

University of São Paulo
Institute of Mathematics , Statistics and Computer Science

**Efficient Development of Multi-Agent
Conversational Systems with Local
and Remote LLMs and the
Model Context Protocol**

Eduardo Figueredo Pacheco

São Paulo, Brasil
2025

University of São Paulo
Institute of Mathematics , Statistics and Computer Science

Efficient Development of Multi-Agent Conversational Systems with Local and Remote LLMs and the Model Context Protocol

Eduardo Figueredo Pacheco

Undergraduate thesis presented to MAC0499, **Computer Science**,
Institute of Mathematics Statistics and Computer Science
University of São Paulo.

Supervisors:
Prof. Fábio Cozman
Prof. Mairieli Wessel

São Paulo, Brasil
December 2025

Acknowledgments

“Se adapte, sobreviva, se modifique, viva.” - Pitty

A line from the Brazilian singer Pitty’s song *“Admirável Chip Novo”* remains a quiet force to me — a reminder to stay human in moments when machines and technology grow ever stronger around us, and to keep choosing the kind of movement and change that only humanity can create.

I thank God for all He has given me, for guiding me, and for protecting me through every challenge, including the trials of Calculus III, Statistics, and the long nights of Computer Networks. It is humbling to look back and see how many of my prayers have already been answered, how many of my dreams have quietly become reality. These moments remind me that I am not alone, and that God’s plans for my life are far greater than my own. For all of this, I am deeply grateful.

My deepest gratitude goes to my family — Marizete, Idelfonso, Léo and João — and to the friends who have become family to me, whether from Vitória-ES, from my years at university, or from São Paulo. Especially to Bianca, Cinthia, Jefferson, and all of those who walked with me throughout this long journey: you believed in me even when I doubted myself, and your presence was essential at every step. You have been, and continue to be, fundamental in my life, and you more than deserve to be here in this small note of gratitude.

I thank all my professors for shaping my path, and when I say professors, I do not refer only to those from my university, but to every teacher who taught me even the smallest things, who helped me look a little farther, who told me I was capable of more, and who believed I could reach places I had never imagined.

There is one memory I carry with particular affection. When I was still in elementary school, a teacher of mine once spoke to a man passing through the hallway and said, “This boy is very intelligent. He can achieve anything he puts effort into.” I was playing with the other children, yet even from a distance I heard those unexpected words. They stayed with me. They shaped the way I saw myself at a time when I did not understand my own potential. And they taught me something I hope never to forget: believing in someone’s

ability, especially when they cannot yet see it themselves, can change the course of their life. That small gesture of trust became a quiet force in mine.

And especially, I would like to thank my advisors Prof. Fábio Cozman and Prof. Mairieli whose guidance made this project possible. I am particularly grateful to Professor Alfredo Goldman for supporting this work not only as a thesis but also as an entrepreneurial project that opened the door to my international scholarship in the Netherlands.

Finally, my sincere thanks to the University of São Paulo for welcoming me, challenging me, and giving me the space to grow.

Abstract

This thesis presents the development of a secure multi-agent conversational system built around the Model Context Protocol (MCP). MCP provides a standardized and controlled way for language models to interact with external functionalities, whether APIs, internal services, or custom servers, allowing organizations to define their own tools and execution boundaries. In this project, both the agent and a custom MCP server were implemented to support enterprise-style workflows.

The system adopts a multi-agent orchestration layer in which dedicated components, such as a planner and tool executor, collaborate to interpret user requests and select the appropriate sequence of MCP tools. This design improves modularity, safety, and determinism by ensuring that all operations occur only through explicit and auditable tool calls.

A hybrid evaluation was conducted using both local LLMs and external API-based models to compare accuracy, latency, and robustness. The results demonstrate the feasibility of deploying MCP-based multi-agent systems in environments with different computational constraints while maintaining strong security and operational control.

Keywords: Multi-Agent Systems, Model Context Protocol, Secure Tool Use, Conversational Agents, LLM.

Contents

Acknowledgments	ii
Abstract	iv
1 Introduction	1
1.1 Context and Motivation	1
1.2 Problem Statement / Research Gaps	3
1.3 Research Questions (RQs)	4
1.4 Thesis Structure	4
2 Conceptual Background	6
2.1 Introduction to Conversational AI	6
2.1.1 Natural Language Processing, Understanding, and Generation . . .	7
2.1.2 Architecture and Components of Modern Conversational AI	7
2.2 Large Language Models	8
2.2.1 Historical Evolution of Language Models	8
2.2.2 The Transformer Architecture	9
2.2.3 Context Management, Inference, and Model Guidance	10
2.2.4 Open-Source and Proprietary Models	11
2.2.5 Current and Future Perspectives of LLMs	13
2.3 Retrieval-Augmented Generation (RAG)	13
2.4 Multi-Agent Systems in Conversational AI	14
2.4.1 Core Concepts	14
2.4.2 Typical Roles	14
2.4.3 Benefits and Trade-Offs	15
2.4.4 Link to Implementation	15
3 Model Context Protocol (MCP)	16
3.1 Introduction	16
3.2 Objectives	18
3.3 Architecture and Core Components	18
3.3.1 Core Components	19

3.3.2	Static Architecture vs. Dynamic Workflow	21
3.3.3	Operational Workflow	21
3.3.4	Local vs. Remote MCP Servers	22
3.4	Applications and Tools	25
4	Project Development	26
4.1	Overview	26
4.2	Initial Research and Planning	26
4.3	Early Prototypes and Experiments	27
4.3.1	RAG-Based and Task-Oriented Conversational Agent	27
4.3.2	Lightweight RAG-Based Conversational Agent with Embedded Ac- tions and Database Integration	29
4.4	Architectural Challenges and Security Risks in Task-Oriented Agents . . .	31
4.4.1	Security Concerns and the Risk of SQL Injection	31
4.5	From Prototypes to the Final Architecture	32
4.6	Final System Architecture	33
4.6.1	Custom Airline API (FlightCo API)	35
4.6.2	MCP Tool Server (Flight Customer Service MCP)	35
4.6.3	Multi-Agent Orchestration Layer	36
4.6.4	Agent API and Communication Interface	37
4.7	Front-End Demonstration and Integration Flexibility	38
4.7.1	Integration With Any Front-End Environment	38
4.7.2	Deployment, Infrastructure, and Development Workflow	39
4.8	Tools, Libraries, and Technologies	40
4.9	System Architecture and Design Strategy	41
4.10	Evaluation Workflow and Lessons Learned	43
5	Methodology	44
5.1	Research and Development Approach	44
5.2	Evaluation Strategy	45
5.3	Benchmark Question Categories	45
5.4	Metrics and Data Collection	46
5.4.1	Batch Execution and Logging Pipeline	46
5.4.2	Accuracy Definition and Annotation Procedure	47
5.4.3	Aggregation of Latency and Token Metrics	48
6	Results	49
6.1	Local LLM Results	49
6.2	API-Based LLM Results	50
6.3	Cost and Resource Considerations	51

6.4	General Observations	51
7	Discussion	52
7.1	MCP and Reliability in Tool Execution	52
7.2	Latency Differences and Token Effects	53
7.3	Cost, Privacy, and Viability of Local Deployment	53
7.4	Implications for Multi-Agent Design	54
8	Conclusion	55

List of Figures

1.1	Comparison between a traditional interface and an agent-based workflow, illustrating how natural language can mediate real actions.	2
2.1	Key components of a conversational AI system.	6
2.2	Historical Timeline of Key Milestones in Language Models	9
2.3	The core structure of the Transformer architecture. The model consists of an Encoder and a Decoder, each with N layers. Self-Attention and parallel data processing are the pillars underpinning most modern Large Language Models (LLMs).	10
2.4	Comparison between a standard LLM (without RAG) and a RAG-based system, showing how external data retrieval enhances factual grounding. .	14
3.1	Comparison of tool integration before and after the introduction of the Model Context Protocol (MCP).	17
3.2	MCP architecture: interaction between the host, client, and servers connected to local or remote data sources.	19
3.3	MCP workflow: discovery, tool selection, invocation, and updates in the interaction between the LLM, the MCP client, and the MCP server.	21
4.1	Simple task-based branching logic to trigger backend operations.	30
4.2	High-level architecture of the final system. The diagram highlights the three-layer organization (User Front-End, Agent Layer, and MCP + API backend), and illustrates the complete information flow from user input to the system's final output. Gold arrows indicate internal communication within the reasoning and execution pipeline, while green arrows denote the final response path back to the user. Communication between modules may occur using structured JSON messages, REST/HTTP requests, or the MCP protocol.	34
4.3	Example of the FlightCo front-end interface executing an MCP tool (flight purchase) through the agent.	38

List of Tables

- 2.1 Approximate pricing for popular API-based proprietary and open-source LLMs. Data sourced from LLM Price Check [1]. 12
- 3.1 Condensed comparison of Local vs. Remote MCP Servers (adapted from [2]). 24
- 6.1 Latency and accuracy for local LLMs. 49
- 6.2 Token statistics for local LLMs. 50
- 6.3 Latency and accuracy for API-based LLMs. 50
- 6.4 Token statistics for API-based LLMs. 51

Chapter 1

Introduction

The rapid evolution of artificial intelligence has reshaped how humans interact with technology, introducing a new era of intelligent systems capable of understanding, reasoning, and acting through natural language. Since the emergence of large language models (LLMs), natural language processing has become a foundation for innovation across industries, empowering products and services that dynamically adapt to user intent.

These models have demonstrated remarkable versatility, from text generation and content understanding to supporting decision-making and task automation. LLMs are driving significant advancements across diverse sectors by automating complex tasks, improving accuracy, and providing deeper insights [3]. At the center of this transformation lies the concept of intelligent agents: autonomous entities designed to interpret user goals and interact with digital environments.

Modern AI agents extend beyond simple chat interfaces, integrating reasoning, memory, and connectivity with external systems. This growing complexity enables them to operate in real-world contexts, performing actions that traditionally required human supervision while maintaining greater transparency and reliability [4].

The recent development of open-source frameworks and communication standards, such as the emerging Model Context Protocol (MCP), has further accelerated experimentation in this area. These innovations enable agents to connect seamlessly with diverse tools and services, fostering fluid collaboration between human instructions and machine execution. Yet, designing systems that balance flexibility, control, and security remains a core technical challenge in this rapidly advancing field.

1.1 Context and Motivation

Traditional language-model architectures, such as Retrieval-Augmented Generation (RAG), have proven effective for enhancing factual question answering and text generation by retrieving information from external sources. However, these systems remain fundamentally limited: they respond to user queries but do not *act* on behalf of the user.

As artificial intelligence advances toward more interactive and operational capabilities, there is a growing demand for agents that can execute actions, automate workflows, and interface directly with real systems through natural language.

The central motivation of this work is therefore to explore how LLM-based agents can simplify and augment the use of digital tools by enabling users to interact with them naturally, without needing to understand interfaces, schemas, or system-specific procedures. Instead of navigating menus, filling forms, or calling APIs directly, users should be able to express their goals in natural language and rely on an intelligent agent to execute the necessary steps in a safe, auditable, and structured manner.

To illustrate this contrast, Figure 1.1 presents a simplified example comparing a traditional interaction flow with an agent-based approach. The “Before” side shows a conventional workflow requiring manual authentication, navigation, and form-filling. The “After” side demonstrates how an agent can perform the same operations through a conversational interface, where the user simply states their intention, such as purchasing or canceling a ticket, and the agent executes the corresponding actions autonomously. This example is adapted from a fictitious airline scenario¹ but is not meant to emphasize aviation itself; rather, it highlights how natural-language agents can streamline interactions across *any* domain.

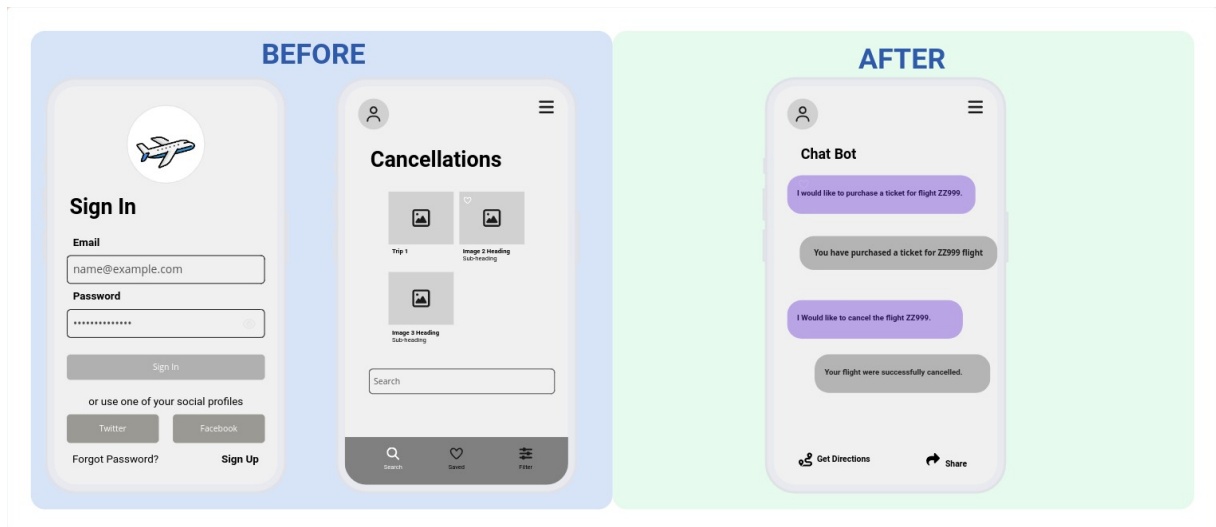


Figure 1.1: Comparison between a traditional interface and an agent-based workflow, illustrating how natural language can mediate real actions.

This project adopts an airline environment solely as a concrete demonstration of the broader vision: intelligent agents capable of interacting with diverse tools, APIs, and company systems in a controlled and structured way. The aviation scenario provides clear examples of operations, booking, canceling, and retrieving data, but the underlying architecture is intentionally generalizable. The same agent design can support tasks in

¹Source: user-provided illustration.

banking, e-commerce, healthcare, logistics, corporate operations, customer support, or any workflow that requires structured interaction with external systems.

To enable such capabilities safely, this work employs the Model Context Protocol (MCP), which provides a robust framework for mediating interactions between agents and external tools. MCP ensures that every action is constrained, validated, and auditable, addressing the security risks identified in early experiments with approaches such as direct Text-to-SQL execution. By relying on tool-based, schema-defined actions, MCP aligns with contemporary agent design trends – including function calling, ReAct-style reasoning, and structured tool use – while maintaining strong guarantees around system integrity [2, 4].

Ultimately, this project investigates how emerging agent technologies can bridge the gap between natural language understanding and practical execution. By enabling agents to perform real actions safely and reliably [2], this work contributes to the evolution of artificial intelligence from passive text generators to active, goal-oriented systems capable of assisting users across a wide variety of domains.

1.2 Problem Statement / Research Gaps

In recent years, numerous tools have emerged aiming to make artificial intelligence more accessible to a wider audience. Many of these platforms allow users to build simple chatbots or automate digital interactions through intuitive, low-code interfaces. While they have expanded the reach of AI technologies, their scope remains limited to predefined workflows and short conversational exchanges, without addressing the complexity required for agents capable of deep reasoning, dynamic decision-making, and secure system integration.

The main challenge explored in this research lies in the implementation of intelligent agents that can operate safely and reliably within complex, real-world environments. Early approaches based on direct database access, such as Text-to-SQL, quickly revealed security risks, a lack of isolation, and limited control over model behavior. Designing agents that interact with external systems without granting unrestricted access requires strict execution boundaries, auditable actions, and clearly defined tool interfaces. The Model Context Protocol (MCP) provides a promising approach by enabling interactions exclusively through isolated tools, ensuring that all operations remain controlled and transparent.

This project examines how different large language models (LLMs), both open-source and proprietary, can be integrated into such an architecture, focusing on their trade-offs in terms of cost, privacy, performance, and reliability. Understanding how to combine multi-agent coordination with MCP-based tools is essential for advancing the development of adaptable and autonomous AI systems capable of acting safely in enterprise contexts.

From an entrepreneurial perspective, addressing these gaps can accelerate innovation and democratize access to intelligent automation. Making advanced, secure AI agents

viable for organizations of all sizes can enhance productivity, reduce operational barriers, and foster closer interaction between businesses and their customers. Moreover, this project emphasizes inclusivity by designing solutions that remain accessible and reliable for diverse users, ensuring that the impact of artificial intelligence extends broadly across society.

1.3 Research Questions (RQs)

This project investigates how multi-agent conversational systems can be designed and executed efficiently using the Model Context Protocol (MCP). The study focuses on architectural design principles and on the practical evaluation of local and API-based language models within this framework. Based on the scope of the work and the experiments conducted, the project is guided by the following research questions:

- RQ1: How can a modular and reliable multi-agent architecture be designed using the Model Context Protocol (MCP)?** This question concerns the organization of planner, FAQ, and tool-execution agents, and the role of MCP in structuring tool invocation, maintaining boundaries, and enabling controlled interaction with external systems.
- RQ2: What are the performance trade-offs between local and API-based LLMs when used in an MCP-driven multi-agent system?** This question examines latency, accuracy, token usage, and operational characteristics across models executed locally versus through external APIs.

1.4 Thesis Structure

This thesis is organized to provide a comprehensive view of the development and study of intelligent agents, particularly focusing on their architecture, operation, and secure integration with external systems. The goal is to progressively build both the theoretical and practical foundations that support the final multi-agent MCP-based solution.

The first chapter introduces the motivation behind this study, outlines the limitations of traditional conversational AI, and presents the early exploration of Text-to-SQL before transitioning to a safer architecture based on APIs and the Model Context Protocol (MCP). Chapter 2 then presents the conceptual background, including conversational AI, large language models, RAG, multi-agent systems, and the relevant technical foundations needed to understand the system.

Chapter 3 explores the Model Context Protocol in detail, discussing its architecture, components, and the role it plays in enabling secure tool-based interactions. Chapter 4 presents the complete development of the system: early prototypes, the identification of

security risks such as unrestricted SQL execution, and the transition toward the final MCP-driven design using the custom FlightCo² API and tool server.

Chapter 5 describes the methodology, including the tools and technologies used, the system architecture, the multi-agent orchestration layer, and the development workflow. It also details the testing procedures, security considerations, limitations, and potential future improvements.

Finally, Chapters 6, 7, and 8 present the results, discuss their implications, and conclude the work.

²FlightCo is a fictitious company created solely for experimentation and evaluation of agent-tool integration, system interoperability, and overall architecture efficiency.

Chapter 2

Conceptual Background

This chapter provides the theoretical and technological foundation that supports the development of modern conversational AI systems. It explores the evolution from simple chatbots to autonomous agents, the mechanisms of Retrieval-Augmented Generation (RAG), and the role of large language models and AI tooling in enabling these advancements.

2.1 Introduction to Conversational AI

Conversational Artificial Intelligence (AI) refers to systems capable of understanding and producing human language in a natural and interactive way. These systems are designed to interpret user intent, manage dialog, and generate meaningful responses, bridging the gap between human communication and machine processing. Over the years, conversational AI has progressed from simple rule-based chatbots to advanced models powered by deep learning and large-scale language models.

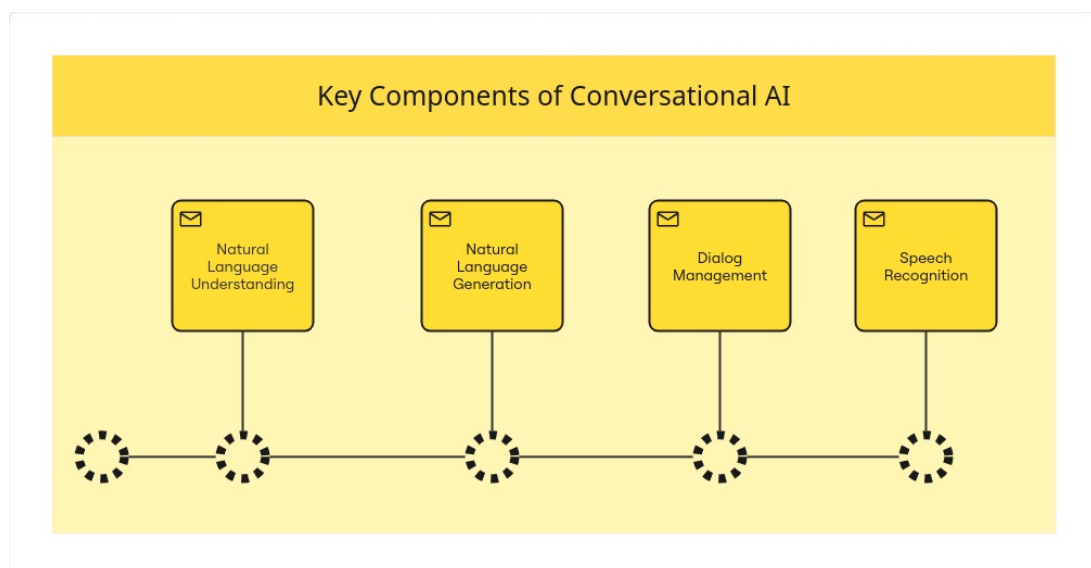


Figure 2.1: Key components of a conversational AI system.

Figure 2.1 summarizes the main components that form the foundation of a conversational AI architecture. Each component plays a distinct role in enabling the system to understand, reason, and respond effectively.

2.1.1 Natural Language Processing, Understanding, and Generation

At the foundation of conversational AI is *Natural Language Processing* (NLP)¹, the field that studies how computers can understand and produce human language. It combines ideas from linguistics, computer science, and machine learning to make interaction with machines more natural and intuitive.

When a user sends a message, the system first needs to understand its meaning, identifying what the person wants and the context of the request. This step, often called language understanding, transforms raw text into structured information that the system can reason about. Once the message is interpreted, the AI generates a response that is coherent, contextually relevant, and sounds natural to the user.

These language processing abilities form the core of modern conversational systems. They allow machines not only to process text, but to participate in meaningful exchanges, a capability that has made NLP one of the most important areas driving recent progress in artificial intelligence.

2.1.2 Architecture and Components of Modern Conversational AI

Modern conversational AI systems are typically organized into interconnected components that work together to create coherent and meaningful interactions. As illustrated in Figure 2.1, these systems follow a modular architecture that separates language understanding, dialog management, and response generation.

In a traditional setup, the process begins with an input. The message is first processed by the language understanding module, which interprets its meaning and identifies the user's intent. The dialog manager then decides how the system should respond, taking into account the current conversation state, user history, and contextual information. Finally, the generation component produces a natural language response, completing the interaction loop.

This modular design provides flexibility and control, allowing developers to fine-tune each component independently. However, it also introduces challenges, such as maintaining consistency across modules and ensuring that context is preserved throughout long or

¹NLP, or *Natural Language Processing*, is a branch of artificial intelligence focused on enabling computers to understand, interpret, and generate human language.

complex conversations.

2.2 Large Language Models

Large Language Models (LLMs) are one of the most important advances in artificial intelligence, transforming how machines understand and generate human language. They are built on the Transformer architecture, which uses large-scale pre-training and attention mechanisms to process information efficiently. Unlike earlier models designed for specific tasks, LLMs can adapt to new problems through context, allowing them to perform a wide range of language-related activities such as translation, summarization, and reasoning without retraining.

Today, there is a broad variety of LLMs developed by different organizations, each with unique strengths and design goals. Some of the most well-known include OpenAI’s GPT models, Meta’s LLaMA series, Anthropic’s Claude, Google’s Gemini, DeepSeek, and Mistral. This section introduces the key ideas behind LLMs, exploring their historical development, main architectural components, and training methods. It also highlights how model scale, data diversity, and fine-tuning strategies have shaped the modern LLM landscape and enabled the creation of these diverse and powerful systems.

2.2.1 Historical Evolution of Language Models

The evolution of Large Language Models (LLMs) reflects a shift from pattern-based systems to highly scalable neural architectures capable of complex reasoning. As illustrated in Figure 2.2, this progression began with early conversational systems such as *ELIZA* (1967), which relied on rule-based pattern matching. A foundational advance occurred with the development of Long Short-Term Memory (LSTM) networks in 1997, enabling models to capture longer-range dependencies and overcome key limitations of recurrent networks.

A second major milestone emerged around 2010 with the introduction of distributed word embeddings, such as Word2Vec and GloVe, which allowed models to learn semantic structure from large corpora instead of relying solely on symbolic or statistical techniques. These representations paved the way for more expressive and data-driven NLP architectures.

The decisive turning point came with the Transformer architecture (2017), which replaced recurrence with a fully parallel self-attention mechanism. This enabled efficient large-scale training and became the foundation of modern LLMs. Models such as BERT (2018) and GPT-3 (2020) demonstrated the effectiveness of this architecture for contextual understanding and generative tasks at scale.

Recent generations of LLMs, such as GPT-4, LLaMA, Qwen, and others released from 2023 onward, extend these capabilities with improved efficiency, multimodality,

and instruction-following behavior. Importantly, many of these models are now open, lightweight, and optimized for on-device or local execution, allowing organizations and developers to run powerful LLMs without relying solely on cloud-based APIs. This shift toward local deployability plays a central role in the architectural choices explored later in this thesis.

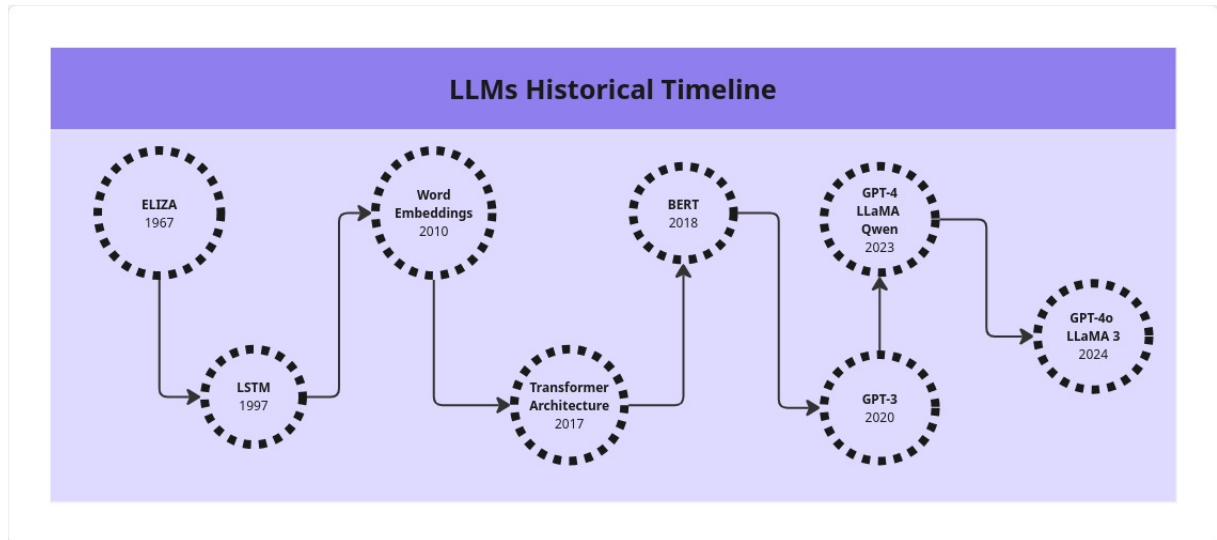


Figure 2.2: Historical Timeline of Key Milestones in Language Models

2.2.2 The Transformer Architecture

The Transformer architecture, illustrated in Figure 2.3, introduced a fully attention-based approach to sequence modeling. Instead of relying on recurrence, it analyzes all tokens in parallel using Self-Attention, allowing the model to capture contextual relationships efficiently. Inputs are embedded, combined with positional encodings, and processed through a stack of encoder and decoder layers.

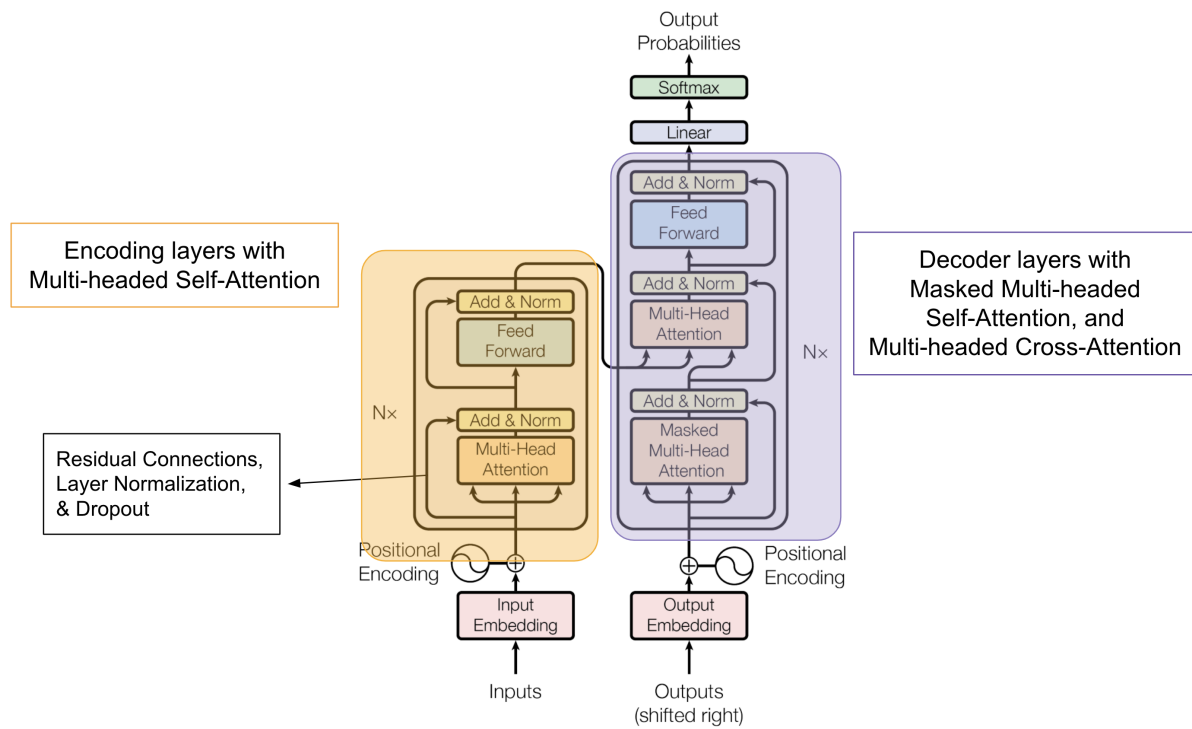


Figure 2.3: The core structure of the Transformer architecture. The model consists of an Encoder and a Decoder, each with N layers. Self-Attention and parallel data processing are the pillars underpinning most modern Large Language Models (LLMs).

The Encoder (orange block) consists of N identical layers, each containing a Multi-Head Self-Attention mechanism and a Feed Forward Network (FFN). Residual connections and normalization stabilize learning and help the model preserve information across layers. This structure allows the encoder to build rich representations where each token is informed by the entire input sequence.

The Decoder (purple block) mirrors this design but includes two key additions: Masked Self-Attention, which ensures that generation remains causal, and Cross-Attention, which lets the decoder attend directly to the encoder's outputs. Together, these components enable the model to integrate input context while producing outputs one token at a time.

The importance of the Transformer lies in its scalability and efficiency. By eliminating sequential processing, it enabled massive parallelization, reducing training time dramatically and allowing models to grow to billions of parameters. This breakthrough made it possible to train powerful LLMs such as GPT, BERT, and T5, and it remains the dominant architecture in modern natural language processing and generative AI.

2.2.3 Context Management, Inference, and Model Guidance

The quality and performance of a large language model depend on several key factors, such as the number of parameters, the diversity and quality of its training data, and the

efficiency of its attention and inference² mechanisms. A higher number of parameters usually allows for richer representations and deeper reasoning, while balanced and diverse datasets improve generalization and reduce bias. LLMs process text through Tokenization³, mapping subword units into high-dimensional embeddings that capture semantic meaning. Self-Attention structures these tokens into contextual relationships, though the context window still limits how much information can be processed at once. Recent methods such as Prompt Engineering and Instruction Tuning help guide model behavior, while scaling laws and optimized architectures contribute to faster, more coherent, and contextually aware outputs.

2.2.4 Open-Source and Proprietary Models

Evaluating the differences between open-source and proprietary large language models (LLMs) is essential for understanding their impact on cost, security, and system design. These distinctions influence how data is processed, where computations occur, and how easily organizations can audit or customize the underlying architecture. Because intelligent agents in this project may operate in sensitive environments, selecting the appropriate class of model, local or API-based, affects both privacy and operational reliability.

Open-source models provide full control over deployment, offering transparency and privacy guarantees but requiring significant computational resources and technical expertise. Proprietary models, in contrast, offer high performance and convenience through managed APIs, at the cost of data leaving the environment and variable usage-based pricing. These trade-offs influence which components of an MCP-based multi-agent architecture can be executed locally and which may benefit from external inference.

The following subsections compare both categories in detail, highlighting their advantages, limitations, and suitability for secure multi-agent systems.

Open-Source Models

Open-source models, such as LLaMA, Mistral, and Falcon, are distributed under permissive licenses (e.g., Apache 2.0) and can be executed entirely within an organization's infrastructure. This local execution guarantees that sensitive data never leaves the internal environment, which is particularly important for enterprise-grade or regulated scenarios. Because their architectures, and sometimes their training datasets, are publicly documented, these models enable auditing for vulnerabilities, bias analysis, and fine-grained customization aligned with internal requirements.

²Inference refers to the process of generating text based on the model's learned parameters and input prompt.

³Tokenization is the process of converting text into discrete numerical units called tokens, enabling efficient handling of vocabulary and context.

The absence of per-token usage fees makes open-source deployment cost-efficient at scale. However, local inference demands high computational capacity, typically requiring GPUs such as the NVIDIA A100, H100, L40, or multi-GPU consumer setups. This need introduces additional costs related to infrastructure maintenance, energy consumption, and engineering expertise. Furthermore, optimizations such as quantization⁴ and model pruning must be handled internally. Open-source models therefore maximize control and privacy but require substantial hardware investment and ongoing technical management.

Proprietary Models

Proprietary models, such as GPT-4o, Claude 3, and Gemini 1.5, are offered exclusively through cloud-based APIs⁵. These providers offer highly optimized inference infrastructure with predictable latency, automatic scaling, and continuous performance improvements. This managed environment eliminates the complexity of maintaining local hardware and ensures access to state-of-the-art capabilities. Many providers comply with international data protection standards such as GDPR, though the requirement to send data externally introduces potential exposure risks and limits full operational control.

Another defining characteristic of proprietary models is their usage-based pricing structure. Instead of purchasing hardware, organizations pay per million input and output tokens processed. This can lead to significant operational costs depending on workload volume. Table 2.1 summarizes typical pricing for widely used API models, illustrating the cost differences between providers and model sizes.

Model	Provider	Context	Input \$ / 1M	Output \$ / 1M
GPT-4o	OpenAI	128K	\$5.00	\$15.00
GPT-4o (Aug 2024)	OpenAI	128K	\$2.50	\$10.00
Claude 3 Opus	Anthropic	200K	\$15.00	\$75.00
Llama-3.1 405B Instruct	Fireworks	128K	\$3.00	\$3.00
Llama-3.1 70B Instruct	DeepInfra	128K	\$0.52	\$0.75
Gemini 1.5 Pro	Google	1M	\$3.50	\$10.50
GPT-4o Mini	OpenAI	128K	\$0.15	\$0.60

Table 2.1: Approximate pricing for popular API-based proprietary and open-source LLMs. Data sourced from LLM Price Check [1].

Even with higher operational costs and reduced control, proprietary solutions remain attractive for organizations prioritizing ease of deployment, high accuracy, and immediate access to advanced models without managing their own infrastructure.

⁴A method that reduces precision to lower memory usage and speed up inference.

⁵Application Programming Interfaces enabling remote execution via standardized requests.

2.2.5 Current and Future Perspectives of LLMs

Current Landscape

In recent years, language models have become increasingly capable and widely adopted. Some of the most used today include OpenAI’s *GPT-4*, Anthropic’s *Claude 3*, Google’s *Gemini*, Meta’s *LLaMA 3*, and Mistral’s open models. A new phase is the connection of models with external tools and systems. Instead of only generating text, they can now perform tasks such as retrieving data, analyzing files, or interacting with APIs. Protocols like the *Model Context Protocol* (MCP) are part of this evolution, allowing models to access local or online information safely and execute actions in real environments.

Future Perspectives

Looking ahead, future models are expected to become more efficient, integrated, and reliable. Research is moving toward systems that can reason across multiple types of data such as text, images, and sound while using fewer computational resources. In the future, LLMs will likely act as central components of intelligent ecosystems, collaborating with other specialized agents to perform complex tasks in a more natural and adaptive way.

2.3 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) allows Large Language Models (LLMs) to enhance their responses by retrieving external information before generating text. Instead of relying solely on internal parameters, the model first performs a semantic search and then produces a more grounded and factual answer. Figure 2.4 illustrates the difference between a standalone LLM and one equipped with a retrieval pipeline.

Although RAG was explored in the early prototypes of this project to enhance contextual accuracy and support domain-specific queries, it quickly became clear that it could not meet the security and action-oriented requirements of a production-ready agent. For this reason, the final architecture does not incorporate RAG. Instead, it adopts an API-driven MCP design in which all operations are executed through explicit and controlled tools. RAG improves informational grounding, but it remains fundamentally reactive because it retrieves relevant content but cannot plan, decide, or safely act within real workflows.

For this reason, the final system does not rely on RAG. The project transitioned to a fully **MCP-based, API-first architecture**, where all operations are performed through explicit, secure tools rather than retrieval pipelines. RAG remains conceptually relevant, but it is not part of the final design.

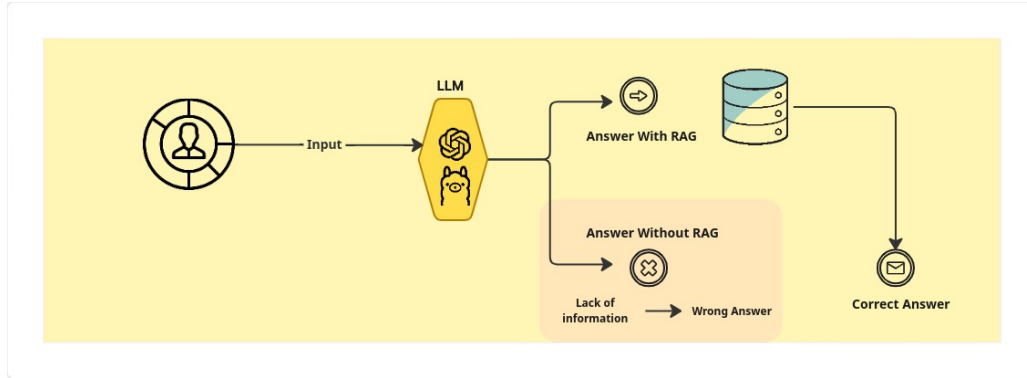


Figure 2.4: Comparison between a standard LLM (without RAG) and a RAG-based system, showing how external data retrieval enhances factual grounding.

2.4 Multi-Agent Systems in Conversational AI

Multi-agent architectures distribute conversational workloads across specialized components rather than relying on a single, monolithic LLM prompt. Each agent encapsulates a distinct function (e.g., planning, tool execution, knowledge response), reducing prompt size, isolating failures, and enabling incremental evolution.

2.4.1 Core Concepts

A Multi-Agent System (MAS) is a set of autonomous agents pursuing a shared objective under partial information. In LLM-driven environments, this yields: (i) specialization (different prompting styles per role), (ii) modularity (replace or extend agents without global refactoring), (iii) traceability (explicit routing decisions), and (iv) guarded side effects (tool use separated from general dialogue).

2.4.2 Typical Roles

- **Planner / Router:** Interprets user intent, selects a route (tool action, login, FAQ) and provides rationale.
- **Tool / Execution:** Invokes external capabilities (e.g., via a protocol specification) with confirmation gating.
- **Knowledge / FAQ:** Answers low-risk informational queries using static or curated text.
- **Memory Layer:** Maintains a bounded interaction window for coherence while controlling token cost.

Agents coordinate through structured message passing and minimal shared state snapshots (e.g., login or confirmation flags), avoiding uncontrolled global context growth.

2.4.3 Benefits and Trade-Offs

- **Benefits:** Modularity, fault isolation, risk segmentation, auditable routing, scalable extension.
- **Trade-Offs:** Added orchestration latency, increased coordination complexity, need for policy layers (security, rate limiting).

2.4.4 Link to Implementation

In the project (`agent/multi`), the `MultiAgentOrchestrator` coordinates a planner, a tool-focused agent (for MCP actions), and a FAQ agent over a windowed memory. This concrete mapping illustrates the theoretical advantages: separation of concerns, explicit routing, and controlled external invocation, without embedding all logic in a single prompt.

In summary, multi-agent design transforms conversational AI from a monolithic interaction loop into a structured, extensible pipeline of specialized reasoning and action components.

Chapter 3

Model Context Protocol (MCP)

The Model Context Protocol (MCP), introduced by Anthropic in late 2024, is a standardized protocol that defines how language models can connect and interact with external tools in a unified way. It provides a common communication layer that replaces the need for platform-specific integrations or manual API wiring, enabling interoperability across different AI systems.

As one of the core components supporting the system developed in this thesis, MCP serves as a foundation for building modular and extensible AI applications. Its design allows language models to discover, describe, and use external tools dynamically, promoting flexibility and scalability in environments that combine reasoning and action. Because of its importance to this work, this chapter presents a detailed overview of MCP’s structure, operation, and role in modern conversational AI architectures.

In the context of this thesis, MCP is not only an enabling technology but the backbone of the final system architecture. All agent–tool interactions, including authentication, flight management, customer operations, event handling, and internal orchestration, are mediated exclusively through MCP tools. This ensures that every action performed by the multi-agent system follows a secure, auditable, and standardized execution path, replacing earlier prototype mechanisms and defining the operational core of the final solution.

3.1 Introduction

The Model Context Protocol (MCP) defines a universal standard for tool integration in AI systems. Inspired by the Language Server Protocol (LSP)¹, MCP enables models to list, describe, and invoke external tools at runtime without relying on platform-specific plugins or rigid API connections. It also supports secure, bi-directional communication, allowing tools not only to respond to model requests but also to send events and updates

¹The Language Server Protocol (LSP) was created by Microsoft to standardize communication between code editors and programming language tools, allowing them to share features such as autocompletion and diagnostics through a unified interface.

back to the host.

Recent surveys highlight that traditional stateless interfaces such as REST² and gRPC³ are not sufficient for the stateful, multi-step reasoning workflows required by modern LLM agents. MCP was introduced precisely to overcome these limitations by introducing session-oriented communication, capability negotiation, and secure tool invocation. This evolution enables richer interactions where models maintain context across multiple steps, negotiate available operations with servers, and execute tools in a controlled and safe manner.

Before MCP, each application or service required its own specific API integration with a language model, as shown in Figure 3.1. This approach created isolated connections, making it difficult to maintain and scale systems that relied on multiple tools. With MCP, these integrations are unified under a single standardized protocol, allowing models to interact with many external tools through one consistent interface. This change greatly simplifies the development and interoperability of AI applications.

Since its release, MCP has evolved into an open and rapidly expanding ecosystem. A growing number of platforms, APIs, and services now expose MCP endpoints, allowing seamless interaction between different systems. Likewise, many LLMs and client applications have begun to support the protocol, enabling them to connect dynamically with external tools and data sources. This continuous adoption across the AI landscape demonstrates MCP’s potential to become a foundational layer for interoperability and tool orchestration, making it a key component to be detailed in this work.

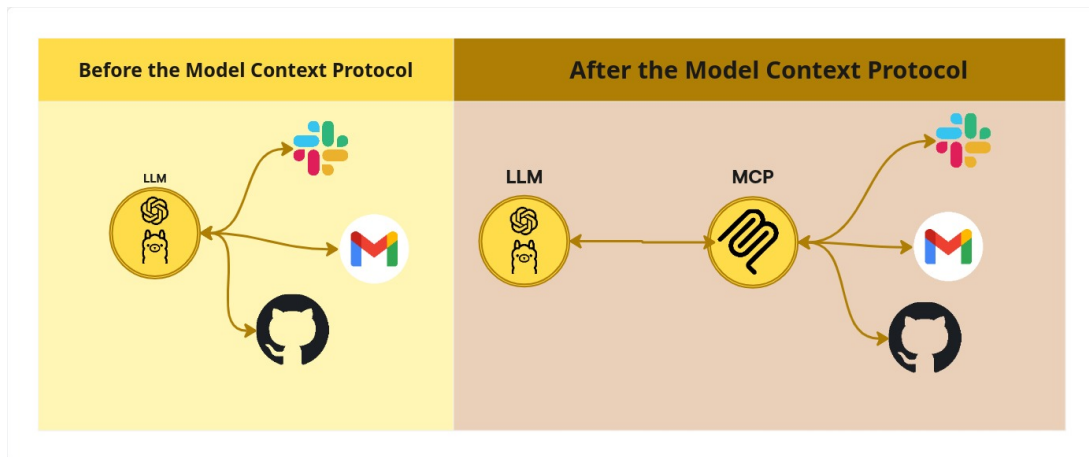


Figure 3.1: Comparison of tool integration before and after the introduction of the Model Context Protocol (MCP).

Within this project, MCP serves as the structural backbone that enables the multi-agent architecture to function reliably. The router, planner, and executor agents all communicate

²REST follows a stateless request–response model, which restricts its ability to support multi-step or stateful agent workflows.

³Although efficient, gRPC also operates under stateless semantics by default, limiting its suitability for long-running, context-preserving agent interactions.

through MCP-based tool definitions, and every operation in the final system, from querying the FlightCo API to executing customer-service workflows, occurs exclusively through MCP tools. By enforcing this single, unified execution layer, MCP ensures isolation, consistency, and security across all agent actions, forming the foundation upon which the entire system operates.

3.2 Objectives

The main objective of the Model Context Protocol (MCP) is to establish a standardized framework that enables AI models to interact with external tools, data sources, and services in a secure, interoperable, and efficient manner. Guided by principles of interoperability, simplicity, extensibility, security, and human-centered control, MCP seeks to unify context management across diverse AI environments and platforms. Its design focuses on minimizing implementation complexity while supporting adaptability to emerging technologies and specialized domains. By providing consistent mechanisms for context sharing, tool invocation, and information exchange, MCP facilitates seamless collaboration between AI systems and external components.

3.3 Architecture and Core Components

The Model Context Protocol (MCP) adopts a modular client-server architecture that separates language models from the external tools, resources, and contextual data they rely on. This separation promotes extensibility and security while enabling LLMs to interact with heterogeneous systems through a unified protocol. Figure 3.2 provides a high-level overview of how these components communicate, emphasizing the flow of information between the host, clients, and servers, as well as the distinction between local and remote execution environments.

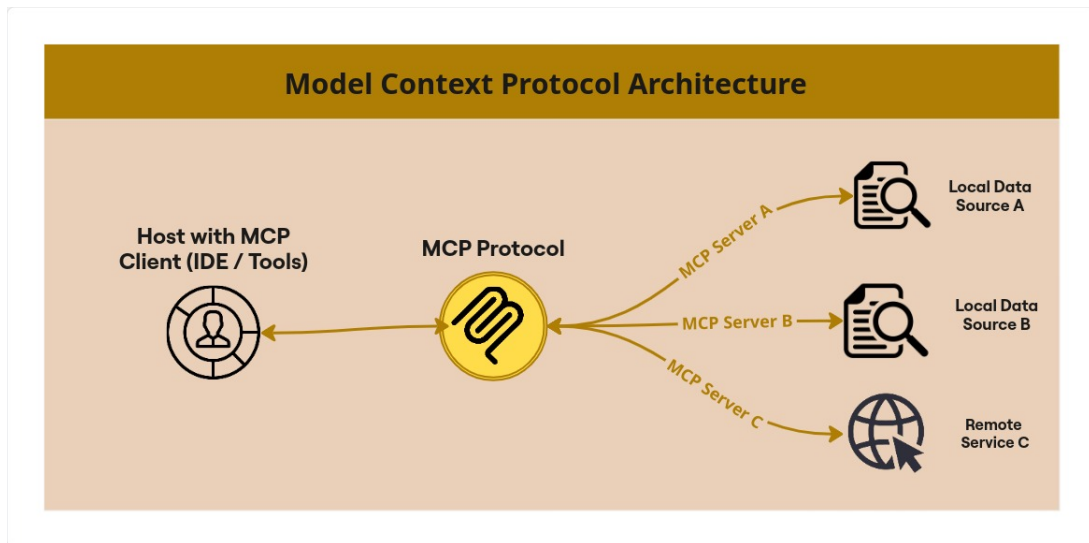


Figure 3.2: MCP architecture: interaction between the host, client, and servers connected to local or remote data sources.

As illustrated in Figure 3.2, the MCP workflow is organized around three core components: the *host*, the *client*, and the *server*. The figure highlights the layered communication pattern: the host orchestrates the system, clients maintain isolated sessions, and servers expose structured capabilities. It also distinguishes how local servers (e.g., filesystem, Git repositories, desktop applications) and remote servers (e.g., APIs, cloud databases, SaaS platforms) integrate into the protocol, allowing LLMs to retrieve context or call external tools in a uniform manner.

The arrows in the figure reflect two important communication directions. Horizontal arrows represent the structured exchange of messages through JSON-RPC—tool calls, prompt retrieval, resource access, and capability negotiation—while vertical arrows represent administrative controls enforced by the host, such as security boundaries, session management, and authorization. The separation of these layers ensures that each component operates with clearly defined responsibilities.

3.3.1 Core Components

Host

The host acts as the central orchestrator of the MCP ecosystem. It is responsible for spawning and supervising client instances, enforcing security policies, and mediating all interactions between clients, servers, and LLMs. The host aggregates context from multiple servers, ensures that only permitted resources are exposed, validates capabilities during initialization, and manages user consent and authorization flows when remote services require OAuth-based access.

In practical terms, the host coordinates the overall execution environment: it maintains

trust boundaries, verifies model requests, controls the lifecycle of each client, and prevents servers from accessing unintended data or interacting outside their negotiated scope.

Client

Each client represents a single session with a specific server. Clients are stateful and isolated, ensuring that requests, resources, and tool invocations are scoped cleanly within each session.

A client is responsible for:

- initiating the session through capability negotiation;
- routing JSON-RPC messages and tracking request state;
- exposing controlled context through *roots*, which define allowed filesystem or data domains;
- mediating LLM sampling requests, including model selection and human approval;
- handling notifications such as file updates, progress events, and capability changes.

In Figure 3.2, clients occupy the middle layer, bridging high-level model reasoning with concrete server capabilities.

Server

Servers expose the functional capabilities that enrich LLM behavior. They may run locally or remotely, but all implement a standardized interface based on four categories:

- **Tools:** executable functions with well-defined schemas (e.g., querying an API, modifying files, executing commands);
- **Resources:** structured or unstructured data accessible via URIs, optionally with change subscriptions;
- **Prompts:** reusable templates that provide consistent scaffolding for model interactions;
- **Utilities:** logging, autocompletion, pagination, and notifications.

Servers operate under strict isolation: they cannot read global host context, communicate with other servers, or escalate permissions. This design prevents cross-domain interference and ensures predictable behavior even in multi-agent or multi-server deployments.

3.3.2 Static Architecture vs. Dynamic Workflow

While Figure 3.2 captures the static organization of the MCP ecosystem, understanding its operation requires a dynamic perspective. During runtime, clients continuously exchange structured messages with servers, discover capabilities, invoke tools, retrieve resources, and receive notifications. The host orchestrates this workflow, ensuring that all interactions respect session boundaries, security policies, and user-defined constraints.

This dynamic behavior, tool invocation, context aggregation, multi-step reasoning, and LLM sampling, is depicted in Figure 3.3 and discussed in detail in the following section, where the operational lifecycle of the protocol is examined step-by-step.

3.3.3 Operational Workflow

To complement the architectural overview, Figure 3.3 presents a step-by-step depiction of the MCP runtime workflow, showing how the *LLM*, the *MCP client*, and the *MCP server* interact throughout the tool-use cycle. The diagram clearly separates the responsibilities of each component and highlights the chronological stages that allow an AI model to discover capabilities, invoke actions, and process updates securely and consistently.

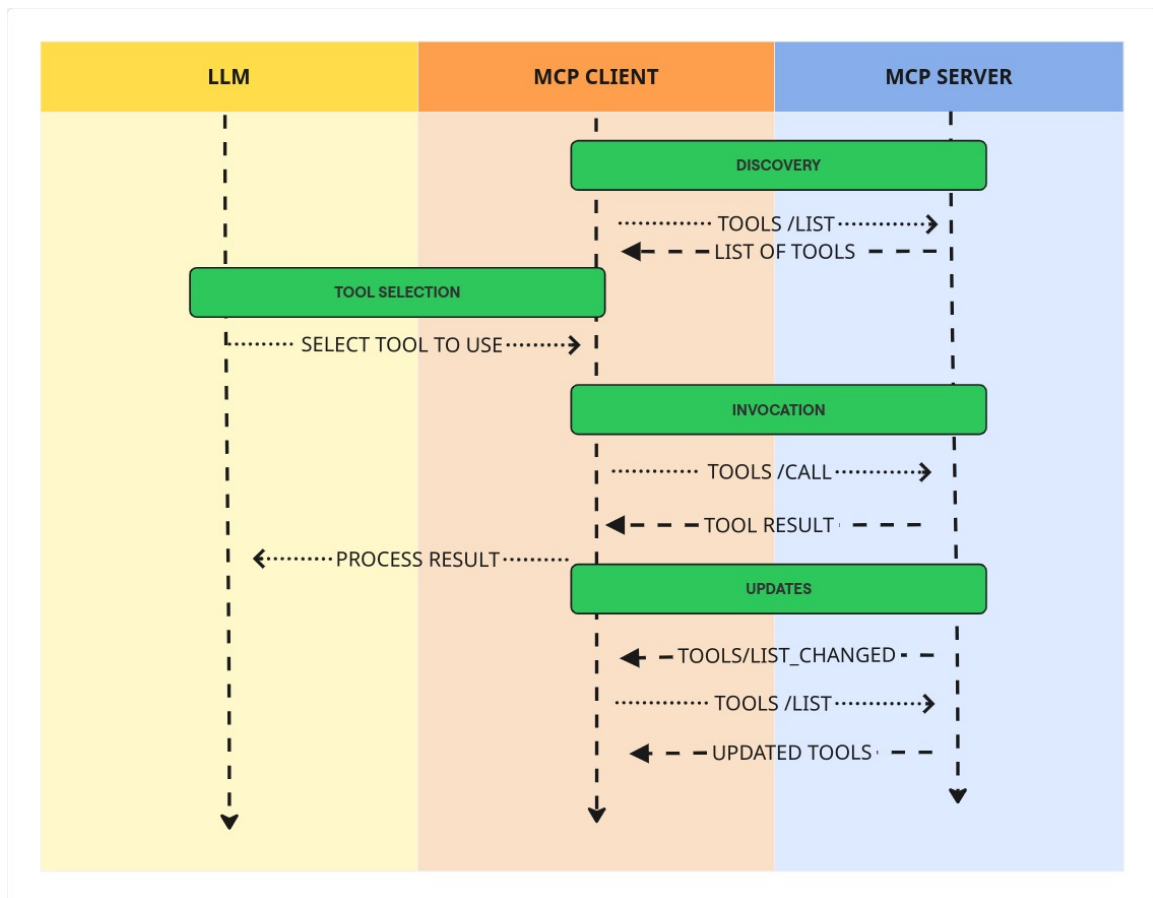


Figure 3.3: MCP workflow: discovery, tool selection, invocation, and updates in the interaction between the LLM, the MCP client, and the MCP server.

As shown in Figure 3.3, the workflow begins with a **discovery** phase, where the MCP client queries the server for a list of available tools. This list contains structured definitions, including input schemas, output formats, and tool descriptions, that allow the model to understand which actions it may safely perform. This capability negotiation phase ensures that the AI model has complete visibility into the external functions it is allowed to call.

Next, the **tool selection** stage occurs within the LLM, where the model reasons over the user’s request and determines which tool is most appropriate for the task. The MCP client then performs the corresponding **invocation**, sending a well-formed request to the server using the standardized JSON-RPC protocol. The server processes the action, e.g., querying a database, executing an API call, or performing domain-specific logic, and returns a structured **tool result** back to the client.

Finally, the workflow supports continuous **updates**, allowing servers to notify clients when tools change, new capabilities become available, or existing tools require re-synchronization. This update mechanism ensures consistency across executions and prevents stale tool definitions from creating errors or unsafe behaviors.

This runtime perspective is essential for the functioning of the multi-agent system developed in this thesis. Every operation executed by the agents, whether involving authentication, flight management, or customer-service workflows, follows exactly the sequence shown in Figure 3.3. By enforcing discovery, structured invocation, and controlled updates, the MCP workflow guarantees that actions remain auditable, predictable, and aligned with the system’s security constraints. This guarantees that the project operates on a reliable execution backbone, where the intelligence of the LLM is combined with strict tool governance mediated by the MCP client and servers.

3.3.4 Local vs. Remote MCP Servers

The Model Context Protocol (MCP) can be deployed through two fundamentally different execution models: *local* servers, spawned directly by the Host on the user’s machine, and *remote* servers, operating as independent networked services. Although both adhere to the same JSON-RPC semantics and capability-negotiation rules, their operational characteristics, performance constraints, and security guarantees differ significantly. Understanding these differences is essential when selecting or designing MCP components for agentic systems. The comparative analysis follows the dimensions outlined by Ray [2], whose survey provides the first system-level benchmark of MCP architectures across real deployments.⁴

⁴See the comparative study of local and remote servers in Table III of the original survey: :contentReference[oaicite:1]index=1

Execution and Transport Differences

Local MCP servers are launched as subprocesses (e.g., via `npx mcp-server-filesystem`), communicating through the `stdio` transport. This channel offers extremely low latency, high bandwidth to the filesystem, and minimal startup overhead. Because communication occurs entirely within the Host’s execution environment, message parsing and transport scheduling are simplified, enabling rapid request–response cycles and high-frequency notifications (e.g., for file updates).

Remote MCP servers, in contrast, operate as standalone HTTP services exposing a single `/mcp` endpoint. Their communication relies on HTTP POST requests and optional Server-Sent Events (SSE) streams for asynchronous updates. This enables multi-tenant access, horizontal scaling, and persistent session management through `Mcp-Session-Id` headers, but introduces network latency, connection instability, and higher operational costs. As Ray highlights, remote servers must incorporate robust error handling, reconnection logic, and health probes to maintain reliability in distributed settings.

Security and State Management

Local servers inherit the Host’s operating-system permissions, typically relying on allow-lists of root paths rather than token-based authentication. While convenient and suitable for personal or on-premise deployments, this model lacks strong boundary enforcement and may expose broader OS-level privileges if misconfigured.

Remote servers, however, implement full OAuth 2.1 with PKCE, dynamic client registration, metadata discovery, and scoped authorization tokens. These capabilities enable secure integration with enterprise identity providers, multi-user settings, and fine-grained access control. Remote servers often maintain durable session state (e.g., RAG caches, user carts) keyed by session identifiers, whereas local servers typically rely on in-memory state that terminates with the process.

Concurrency and Fault Tolerance

Local servers generally handle a single client—the spawning Host—simplifying concurrency and reducing the risk of race conditions. Upon failure, Hosts simply restart the subprocess.

Remote servers must handle large numbers of concurrent MCP clients, overlapping HTTP requests, and asynchronous SSE pushes. They require structured recovery mechanisms such as exponential backoff, request retries, and health-check endpoints (e.g., `/healthz`). According to Ray’s survey, network instability, transport-layer latency, and session-resumption behavior (via SSE `Last-Event-ID`) represent major operational challenges for remote MCP deployments.

Summary Table

Table 3.1 synthesizes the key distinctions between local and remote MCP servers, condensing the extended comparison presented in Ray’s survey (2025).

Table 3.1: Condensed comparison of Local vs. Remote MCP Servers (adapted from [2]).

Aspect	Local MCP Server	Remote MCP Server
Startup & Transport	Spawned by Host using <code>stdio</code> ; minimal latency.	Independent HTTP service; uses POST + SSE; higher network cost.
Security Model	Inherits OS permissions; allow-listed roots; no OAuth.	OAuth 2.1 + PKCE; multi-tenant isolation; scope-based access.
State Handling	Mostly in-memory; tied to process lifecycle.	Durable state keyed by session ID; supports resumption.
Concurrency	Typically single-client; simple locking.	Multi-client; concurrent requests + asynchronous SSE.
Failure Recovery	Host restarts subprocess on crash.	Requires retries, backoff, health probes.
Typical Use Cases	Local filesystem tools, Git repos, desktop automation.	SaaS APIs, enterprise databases, organization-wide knowledge bases.

Integration with MCP Core Components

The distinction between local and remote execution models directly affects how each MCP component behaves:

Host. The Host benefits from low-latency introspection of local servers but must enforce stricter permission boundaries. For remote servers, the Host assumes responsibility for token management, session negotiation, and SSE reconnection strategies.

Client. Clients interacting with local servers operate in tightly-controlled sessions with deterministic state. By contrast, clients of remote servers must manage session identifiers explicitly, handle timeouts, and reconcile asynchronous messages.

Server. Local servers excel at exposing OS-level capabilities (filesystem, Git, local databases). Remote servers specialize in secure, scalable, organization-level integrations (SaaS APIs, cloud datasets, enterprise pipelines). Ray’s survey emphasizes that remote servers must maintain strict capability negotiation and rate limiting to prevent privilege escalation in multi-tenant contexts.

Implications for System Design

In practice, hybrid architectures—combining local and remote servers—provide the best balance between security, performance, and functionality. Latency-sensitive operations (filesystem navigation, code introspection) are delegated to local servers, while high-value or regulated data access (corporate APIs, cloud search, privileged resources) is handled by remote servers with stronger guarantees. This hybrid approach mirrors patterns observed in large-scale MCP agents evaluated in Ray’s study [2], where performance bottlenecks arise primarily from transport latency and OAuth negotiation, while local components deliver near-real-time context for LLM-driven reasoning.

3.4 Applications and Tools

In the MCP framework, tools are the main mechanism that allows AI models to perform actions or access external information. They can represent API calls, database queries, file operations, or other executable functions that extend the model’s abilities beyond its internal context. Each tool is defined in a standard format with a clear description, input parameters, and return values, making it easy for different systems to understand and use them consistently.

Through these tools, MCP enables AI applications to connect with diverse services such as data repositories, analysis platforms, and automation systems. For example, a model may use a tool to fetch data from a local database, summarize a document, or send a query to an external API. This standardized design reduces the complexity of integrating new functions and ensures that tools can be discovered, described, and invoked in a uniform way across different environments.

In practice, MCP tools are implemented on servers that expose external data or functions through the protocol interface. The specific implementation may vary depending on system requirements, but the goal remains the same: to provide a reliable and secure bridge between AI models and the external world, supporting dynamic interaction and flexible task execution.

Chapter 4

Project Development

4.1 Overview

This chapter describes how the proposed conversational agent system was conceived, iterated on, and ultimately implemented. The development process followed an incremental and exploratory approach: starting from an initial survey of existing solutions, moving through a series of prototypes, and converging on a production-inspired, MCP-based multi-agent architecture.

From the outset, the core intention was to build a tool-enabled conversational agent capable of automating customer service workflows, handling structured queries, gated actions (e.g., login, confirmations), and informational fallbacks. Customer service was chosen as a concrete, high-frequency use case, but the resulting multi-agent architecture (planner, tool executor, FAQ fallback, scoped memory) remains domain-agnostic and could be repurposed for technical support, internal operations, education, or e-commerce. This focus guided design decisions such as introducing explicit login state, confirmation handling before sensitive tool invocations, a lightweight fallback knowledge layer, and a bounded context window to control cost while preserving coherence.

As early experiments revealed severe security and reliability issues when connecting LLMs directly to data stores, the project progressively shifted away from direct SQL or raw tool access toward a layered architecture. The final design isolates the agent from databases, routes all operations through an API layer, and uses the Model Context Protocol (MCP) to expose tools in a controlled, inspectable manner. This evolution is reflected in the prototypes and architectural refinements described in the following sections.

4.2 Initial Research and Planning

At the outset of the project, a comprehensive study was conducted to understand the current state of conversational agents. This initial investigation aimed to contextualize

the technological landscape, assess the functionalities and objectives of existing agents, and identify key areas for improvement.

The following aspects were analyzed:

- **General functionality of agents:** How current agents operate, their design goals, and the problems they address across different sectors.
- **Application domains:** Where and how these agents are deployed in real-world scenarios, including industry-specific use cases such as customer service, healthcare, and education.
- **Technological advancements:** The evolution of the technologies enabling these systems, particularly the integration of large language models (LLMs) and modular architectures.
- **Adoption by industry:** Identification of major platforms and companies that utilize or develop conversational agents (e.g., Google, Amazon, Microsoft), and their respective implementation strategies.
- **Identified gaps:** Limitations and open challenges found in existing solutions, helping guide the direction of this project in proposing enhancements.

This research was supported by the theoretical framework presented in Chapter 2, particularly in Sections 2.1 (Introduction to Conversational AI) and Chapter 2.2 (Large Language Models).

A focused study was also conducted on the Model Context Protocol (MCP), discussed in detail in Chapter 3. This included understanding how conversational agents can connect to the MCP, how context is managed and propagated across modules, and the benefits this architecture brings in terms of modularity and scalability.

4.3 Early Prototypes and Experiments

From the beginning of the project, the development of custom agents was a central focus. This enabled practical experimentation with different configurations and validated the assumptions formulated during the initial research phase. These early prototypes laid the technical and conceptual foundation for the more sophisticated systems presented in the subsequent chapters.

4.3.1 RAG-Based and Task-Oriented Conversational Agent

The first functional prototype implemented in the project was based on a Retrieval-Augmented Generation (RAG) architecture, as introduced in Section 2.3. This version was

designed to act as a general-purpose conversational agent capable of handling customer service tasks such as:

- answering frequently asked questions (FAQs);
- providing contextual guidance to users based on their queries;
- executing prompt-based actions in a flexible, interpretable manner.

Even this first agent was built to support multiple large language models (LLMs), such as ChatGPT, LLaMA, and DeepSeek, among others. Although the architecture was model-agnostic, the initial deployments used the GPT-4 Mini API for its balance between performance and cost-efficiency.

A key strength of this prototype was its customizability. It allowed:

- easy configuration of different prompt structures tailored to specific workflows or domains;
- modular integration with external data sources for retrieval;
- personalization of behavior and tone depending on the user profile or company requirements;
- compatibility with any LLM accessible via API, offering high adaptability.

While the system demonstrated strong capabilities in providing helpful answers and guiding users through structured interactions, it still had several limitations. For example:

- it lacked long-term memory or state preservation between interactions;
- the prompt logic was manually crafted and could become difficult to scale or generalize;
- decision-making and task execution mechanisms were entirely embedded in prompt instructions, limiting modularity and robustness.

Despite these limitations, this prototype served as an essential milestone. It validated the core concepts of the system and demonstrated that a single agent could combine RAG techniques with task execution logic to deliver a functional and adaptable solution for customer support contexts. These early lessons informed the subsequent iterations, which focused on improving modularity, expanding capabilities, and integrating the agent more tightly with the Model Context Protocol.

4.3.2 Lightweight RAG-Based Conversational Agent with Embedded Actions and Database Integration

As an early experimental prototype, this version of the conversational agent expanded upon the initial architecture discussed in Section 4.3.1, introducing lightweight task execution and basic database integration. This system was intentionally designed as a proof of concept and does not form part of the final architecture, which later transitioned to a secure MCP- and API-based design. In this prototype stage, the focus remained on exploring the feasibility of simple automated actions within a RAG-driven workflow.

Building on the previous experiment, this version maintains a Retrieval-Augmented Generation (RAG) structure but adds concrete functionalities such as real-time stock verification and simple purchasing actions, which are handled entirely through programmatic logic. Unlike the first prototype, which relied more heavily on prompt engineering and abstract task representations, this iteration implements explicit operational pathways through embedded Python functions. These additions allowed the agent to move beyond purely conversational interactions and actually **perform small, controlled operations** based on user intent.

System Overview

This prototype was built using the LangChain framework, combined with OpenAI’s GPT models (specifically, `gpt-3.5-turbo`) and FAISS for vector-based semantic retrieval. The system components included:

- a custom dataset of frequently asked questions (FAQs) embedded using OpenAI’s embedding API and indexed with FAISS;
- a conversational RAG chain using `ConversationalRetrievalChain` for context-aware, retrieval-augmented responses;
- a lightweight SQLite database used solely for prototyping inventory management and basic transactional logic;
- manually implemented intent-detection logic responsible for triggering predefined actions such as purchasing and stock checking.

As with all early prototypes, this implementation served to explore feasibility, and was later replaced by the final MCP-based architecture, which provides significantly stronger guarantees of security, isolation, and robustness.

Listing 4.1: Manual integration of actions based on user input

```
1 if "buy" in user_input.lower():
2     product_id = 1
3     quantity = 1
4     db_response = buy_product(product_id, quantity)
5     answer = db_response
6
7 elif "how_many" in user_input.lower() and "products" in
8     user_input.lower():
9     product_id = 1
10    stock = check_stock(product_id)
11    answer = f"Available_Products: {stock}" if stock else "No
12            Product"
```

Figure 4.1: Simple task-based branching logic to trigger backend operations.

Features and Capabilities

The second prototype introduced the following improvements:

- **Embedded task execution:** unlike the previous version where task execution was entirely prompt-based, this agent directly executed actions through Python functions, e.g., verifying inventory or processing a purchase;
- **Database integration:** a SQLite backend enabled simple stateful actions such as reading and writing inventory data;
- **Basic intent detection:** simple keyword-based matching was used to infer user intent and route the input to relevant backend logic;
- **Improved RAG responses:** the use of LangChain's `ConversationalRetrievalChain` enabled better context tracking and more accurate information retrieval from the embedded FAQ base.

Limitations and Observations

While this version represents a functional evolution in the agent's capabilities, the overall results were modest. The system has a relatively easy architecture, especially with tools like LangChain and SQLite. However, several limitations persisted:

- **Poor scalability:** hardcoded logic for action routing and SQLite's simplicity make the solution unsuitable for large-scale or complex use cases;
- **Lack of generalization:** new tasks must be manually implemented with dedicated logic, limiting extensibility and increasing maintenance overhead;

- **High token inefficiency:** since the system lacks optimized context windows and tool routing strategies, it can be inefficient in token usage, especially when scaled;
- **Limited tooling:** while some basic tools (purchase, stock check) were implemented, extending functionality would require substantial development effort.

Reflection and Next Steps

Despite offering more actionable capabilities than the initial version, this second prototype showed that while it is simple to implement a working RAG-based assistant with basic tool usage, it is far from production-ready in terms of robustness, modularity, and scalability. The architecture reinforces an important insight: **it is easy to build such agents, but hard to make them scalable, efficient, and worth maintaining**. These lessons directly motivated the transition to a multi-agent, MCP-based architecture with strict separation of concerns and secure tool execution.

4.4 Architectural Challenges and Security Risks in Task-Oriented Agents

After developing the second version of the agent, it became increasingly clear that a more robust code architecture was required. While the implementation provided basic task execution and conversational retrieval, its limitations were evident during practical use. The agent often failed to provide correct answers when given limited or ambiguous context, frequently hallucinated responses, and struggled to scale in complexity and maintainability.

The project highlighted a critical insight: despite how deceptively simple it is to build a working prototype of a task-oriented agent, making it production-ready is a significantly more complex endeavor. Efficient use of tokens, support for dynamic and scalable tool integration, and proper separation of concerns all require a well-structured software design, modular layers, and error handling logic that the initial prototypes lacked.

4.4.1 Security Concerns and the Risk of SQL Injection

Among the most pressing concerns encountered was the insecure way in which tools, such as stock verification and product purchasing, were implemented. The agent was granted direct access to the underlying database via hardcoded SQL queries within its logic. These were triggered based on simple keyword detection in user inputs, with no layers of authentication, parameter validation, or query sanitization.

This opens the system to serious vulnerabilities, particularly SQL Injection Attacks (SQLIA). If the agent were to be deployed in a real-world environment, a malicious user could craft inputs that manipulate the query logic to access or corrupt sensitive data.

The danger is not hypothetical: SQLIA is one of the most common and dangerous web vulnerabilities today.

Recent research emphasizes the critical threat posed by such insecure implementations. In the study by Irungu et al. (2023), the authors explain that SQL Injection remains among the most prevalent cyber-attacks in modern applications, particularly due to a lack of proper input validation. They demonstrate that even tautology-based attacks (e.g., injecting always-true conditions like `WHERE 1=1`) can bypass authentication and manipulate database operations. The authors further propose using supervised machine learning to detect and block SQLIA, achieving detection accuracy rates over 98% [5].

Such findings reinforce the need for strict input validation, parameterized queries, and separation between natural language interfaces and backend execution. Allowing a language model to directly interface with a SQL database via raw queries is inherently insecure and should be avoided in production environments.

This realization informed the direction for the next stages of development, which aimed to introduce a safer, more modular design, where the agent interfaces with well-defined APIs rather than manipulating data sources directly. Tool execution should be governed by controlled, authenticated layers rather than raw user intent interpretation.

Quote from the study: “The SQLIA can be described as a software vulnerability caused by the absence of proper input validation [...] resulting in unauthorized access to the database” [5].

4.5 From Prototypes to the Final Architecture

The previous prototypes demonstrated that building simple task-oriented agents is easy, but making them safe, modular, and production-ready is not. Their limitations, such as manual prompt logic, fragile decision paths, lack of extensibility, and insecure direct database access, made it clear that a new architecture was required.

For this reason, the project shifted its focus to the real goal: a secure, MCP-based multi-agent system designed with proper separation of concerns, robust tool execution, and reliable orchestration between planning, tool routing, and reasoning, whether running entirely locally or integrated with external LLM APIs. The new architecture introduces three key principles:

- **API-first actions:** all operations are implemented as validated HTTP endpoints rather than raw SQL queries;
- **MCP-mediated tools:** tools are exposed through a dedicated MCP server, which defines clear schemas, capabilities, and error surfaces;

- **Multi-agent orchestration:** separate agents handle planning, FAQ responses, and tool execution, preventing a single model from having unrestricted control over the whole stack.

These principles guided the design of the final system architecture, described next.

4.6 Final System Architecture

The final solution developed in this project is a complete, production-inspired ecosystem for experimentation, composed of a multi-agent orchestration layer, a fully custom airline reservation API, a set of MCP-compliant tool servers, a modular LLM execution layer supporting both local and remote providers, and an interactive front-end for user interaction. The system behaves as a realistic enterprise environment for an airline company (FlightCo), enabling operations such as searching for flights, buying tickets, processing cancellations and refunds, updating user credentials, and performing administrative actions.

Before describing each subsystem in detail, it is useful to highlight the main technologies supporting the implementation. The core stack includes Python for all backend components, FastAPI and Uvicorn for HTTP services, MongoDB (via the Motor async driver) for persistent storage, LangChain and LangChain MCP adapters for agent orchestration, and Docker Compose for containerized deployment. LLMs can be executed either locally via Ollama (e.g., LLaMA and Mistral models) or through APIs such as OpenAI and Groq, allowing experiments across a heterogeneous model ecosystem.

Figure 4.2 provides a consolidated view of this architecture. The color-coded regions visually distinguish the three computational layers of the system: the **User Front-End** (yellow), where input queries originate; the **Agent Layer** (orange), where planning, intent classification, FAQ handling, and tool-based reasoning occur; and the **MCP + API Backend** (blue), which executes deterministic operations, communicates with the FlightCo API, and handles persistent data through MongoDB. Gold arrows represent internal communication between agents, LLMs, the MCP server, and the airline backend, while green arrows indicate the final outward-facing response delivered back to the user. These communication paths may involve structured JSON messages, REST/HTTP requests, or the MCP protocol, depending on the specific subsystem involved.

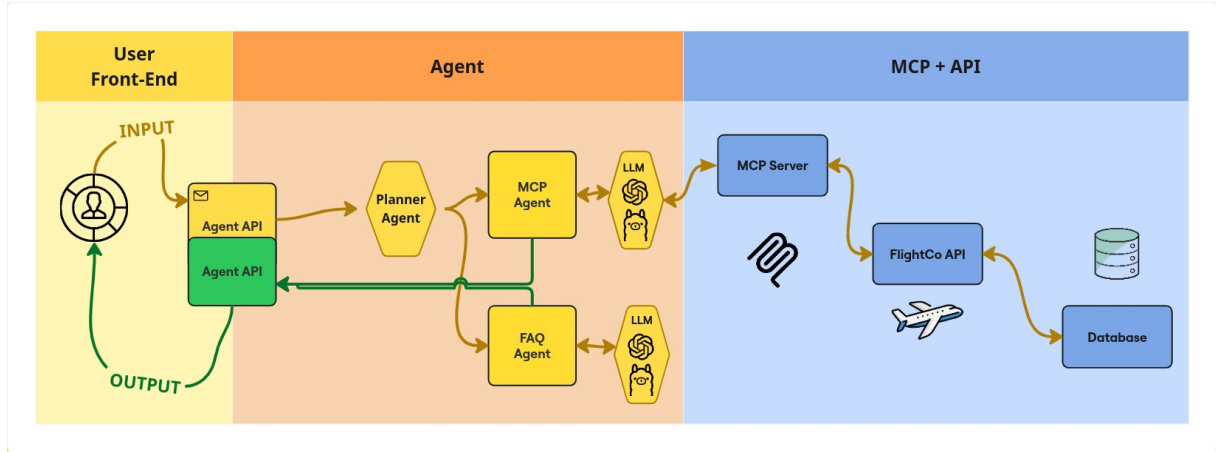


Figure 4.2: High-level architecture of the final system. The diagram highlights the three-layer organization (User Front-End, Agent Layer, and MCP + API backend), and illustrates the complete information flow from user input to the system’s final output. Gold arrows indicate internal communication within the reasoning and execution pipeline, while green arrows denote the final response path back to the user. Communication between modules may occur using structured JSON messages, REST/HTTP requests, or the MCP protocol.

Overall Information Flow and Layer Separation

When the user submits a natural-language request via the front-end, the message is handled by the **Agent API**, which forwards it to the **Planner Agent**. The planner analyzes the request and determines whether it is:

- a **FAQ-level query** requiring no access to internal systems, such as baggage rules, seat-change policies, airline regulations, refund rules, or general company information, which is solved entirely by the **FAQ Agent** using an LLM backend; or
- an **operational request** requiring interaction with FlightCo’s backend (e.g., buying tickets, checking reservations, authentication, cancellation, refunds), which triggers a structured tool invocation via the **MCP Agent**.

This division is crucial: FAQ-level requests remain entirely within the agent layer and never interact with sensitive data, while operational requests involve deterministic tool execution, parameter validation, error propagation, and strict coordination with the MCP server and the FlightCo API. Once the reasoning or tool execution concludes, the **Agent API** assembles the final message and returns it to the user, represented by the green response arrows in Figure 4.2.

The following subsections present each major component of the system and explain their responsibilities in detail.

4.6.1 Custom Airline API (FlightCo API)

A major component of this project is the fully custom REST API built from scratch to simulate an airline’s operational backend. This API is implemented using **FastAPI**, **Motor** (MongoDB async driver), and **Pydantic** for data validation. It exposes a complete stateful reservation system, including user authentication, flight catalog management, ticket purchasing, cancellations, refunds, and audit logging.

The API follows a classical microservice-inspired structure:

- **Authentication Service** (`/auth/...`) supports login, logout, token generation, password changes, and privilege verification;
- **Flight Service** (`/flights/...`) lists available flights, retrieves detailed flight metadata, and manages the flight catalog;
- **Customer Service Operations** (`/cs/...`) exposes endpoints for viewing user reservations, canceling tickets, applying refunds, and querying tickets by status.

All endpoints use JSON for requests and responses. Sensitive operations require JWT-based authentication, enforced through header validation (**Authorization: Bearer <token>**). The API also implements:

- **CORS middleware** for controlled front-end access;
- **Granular access control** (admin, customer service, regular user);
- **Server-side validation** for IDs, e-mails, input length, and regex-based filtering;
- **Comprehensive error handling** with standardized debug messages and HTTP status codes;
- **Audit logs** stored in MongoDB to track administrative actions.

Flight availability is dynamically computed by querying active tickets in the `purchased_flights` collection, ensuring accurate real-time seat counting. All temporal data is stored in UTC following ISO 8601 formatting. The API supports HTTP **GET**, **POST**, and **DELETE** methods and exposes ports typically in the 8000–8003 range, depending on deployment.

4.6.2 MCP Tool Server (Flight Customer Service MCP)

To enable secure tool execution and controlled access to the API, the project includes a dedicated MCP server built using the **FastMCP** framework. This server acts as a middle layer between the agent and the airline backend, exposing deterministic tools mapped to API operations.

Each tool is defined as an asynchronous Python function decorated with `@app.tool()`, automatically registering it in the MCP protocol. Tools include:

- `login()`, `logout()` and credential management;
- `list_available_flights()`;
- `get_flight_details()`;
- `search_customer_flights()`;
- `search_flights_by_status()`;
- `cancel_customer_flight()`;
- `change_password()`.

The MCP server handles:

- **Persistent token storage** using a `tokens.env` file with overwrite-safe update logic;
- **Streamable HTTP transport** for remote servers;
- **Stdio transport** for local execution;
- **Automatic JSON serialization** and pretty-printing of tool outputs;
- **Error propagation and traceback inclusion** for debugging.

Each tool communicates with the API using `httpx.AsyncClient`, passing headers, parameters, and payloads as validated JSON documents. If the API returns a non-200 status code, the MCP tool formats an explicit failure message visible to the agent. This design ensures strict separation between natural language reasoning and backend execution.

4.6.3 Multi-Agent Orchestration Layer

The reasoning core of the system is a **multi-agent architecture** capable of separating high-level planning, tool selection, and natural-language reasoning. This layer is implemented in the `agent/multi` module.

The architecture consists of:

- **Planner Agent**: interprets user requests, classifies intent, selects tools, and builds execution plans;
- **FAQ Agent**: handles simple, public information queries that do not require backend access. Examples include baggage limits, refund policies, seat change rules, and other airline regulations;
- **MCP Agent**: executes MCP tools, handles argument mapping, error checking, and output normalization for any request that requires backend interaction;

- **LLM Interaction Agent:** interfaces with the selected LLM backend (OpenAI, Groq, LLaMA, DeepSeek, Gemini, or Ollama).

Agents communicate using structured messages based on the LangChain `HumanMessage` and `SystemMessage` format. Tool invocation flows follow:

1. The user sends a natural-language query to the Agent API.
2. The Planner decides whether the task is FAQ-based or requires MCP tool execution.
3. If required, the MCP agent calls the corresponding tool with validated arguments.
4. Tool output is formatted and returned to the LLM agent for final response synthesis.

The system supports both fully local execution (via Ollama) and remote LLM APIs. All models return standard JSON-L messages internally to facilitate debugging and deterministic logging.

4.6.4 Agent API and Communication Interface

The multi-agent system is exposed through an HTTP API built with `FastAPI`. The API opens endpoints such as:

- `GET /health` — liveness probe returning `"status": "ok"`;
- `POST /message` — main endpoint used by the front-end.

The `/message` route accepts:

- natural-language user queries;
- a boolean flag `confirm_sensitive` for automatic confirmation of state-changing actions.

The API then:

1. forwards the message to the agent's `run()` method;
2. detects confirmation prompts and automatically continues the flow if allowed;
3. returns the final structured response as JSON.

Static files from the front-end are mounted under `/front`, enabling a complete conversational interface directly from the same FastAPI server. CORS is enabled during development, and all responses include standardized JSON fields such as `reply`, `error`, and `auto_confirmed`.

4.7 Front-End Demonstration and Integration Flexibility

To illustrate how users interact with the agent system, this section presents a real example of the graphical interface used during development. Although a minimal web client was implemented and served directly by the FastAPI backend, the system was intentionally designed to remain decoupled from any specific interface. This allows the conversational agent to operate seamlessly across different environments, such as web dashboards, mobile apps, command-line tools, enterprise portals, or even third-party customer support systems.

Figure 4.3 shows a screenshot of the FlightCo assistant performing a complete tool-enabled workflow, including intent recognition, execution confirmation, and a structured MCP tool invocation.

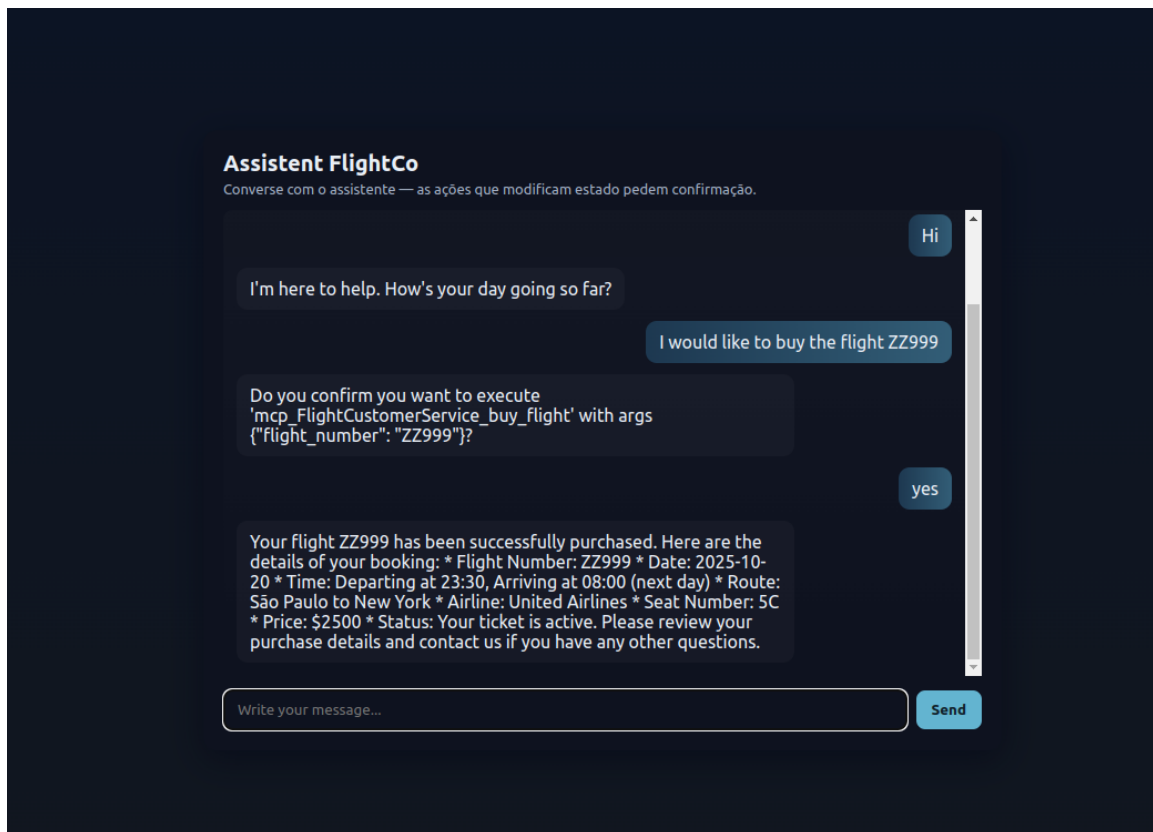


Figure 4.3: Example of the FlightCo front-end interface executing an MCP tool (flight purchase) through the agent.

4.7.1 Integration With Any Front-End Environment

A core design principle of the system is that the conversational agent must be **front-end agnostic**. The agent is not tied to any graphic interface or framework; instead, it exposes a clean and predictable HTTP API that any client can communicate with.

The FastAPI service provides a well-defined endpoint:

- **POST /message** — the primary conversational endpoint used by all interfaces.

Clients send a JSON payload containing the user message (and optional flags such as `confirm_sensitive`), and the server responds with a structured JSON object containing the agent’s reply and metadata. Because this communication occurs over standard HTTP, the system can be integrated using many mechanisms, such as:

- **cURL requests from a terminal**, e.g.:

```
curl -X POST http://localhost:8001/message \
  -H "Content-Type: application/json" \
  -d '{"message": "List my flights"}'
```

- **JavaScript front-ends**, using `fetch()` or `Axios`;
- **Mobile applications**, through native network libraries;
- **Internal enterprise chat interfaces**, such as support panels or CRM dashboards;
- **Third-party chatbot platforms**, which can forward user messages to the API and display the returned response.

This architectural choice ensures that the conversational intelligence and MCP-based tool execution remain centralized in the backend, while the user interface can evolve freely. Companies can create interfaces tailored to their brand, security requirements, aesthetic preferences, or platform constraints without rewriting any part of the agent’s reasoning or tool orchestration layers.

The decoupled design also simplifies experimentation: multiple distinct front-ends may interact with the same agent simultaneously, allowing A/B testing, usability comparisons, or integration with several operational systems within an organization.

In summary, the system’s reliance on standard HTTP communication and JSON-based message envelopes makes the agent highly portable. Any environment capable of performing an HTTP POST request can become a fully functional interface for the AI agent, enabling flexible deployment across web, mobile, desktop, automation pipelines, or embedded systems.

4.7.2 Deployment, Infrastructure, and Development Workflow

The system includes infrastructure components located in the `infra/` and `terraform/` directories. These components provide:

- **Docker Compose** services for the agent, API, MCP server, MongoDB, and front-end;
- **Environment variable management** using `.env` files for reproducibility and isolation across environments;
- **Terraform modules** for provisioning cloud environments and configuring networking, compute instances, and container registries.

Initially, services were started manually in separate terminals (API, agent, MCP server, database), which was sufficient for small experiments but quickly became error-prone as the number of components grew. The introduction of unified `Dockerfiles` and a single `docker-compose.yml` file automated the full stack bootstrap. Auxiliary shell scripts (such as `start.sh`) further simplified initialization, log inspection, and teardown. This evolution turned the project into a fully orchestrated platform that can be launched with a single command on any compatible machine.

Containerization was essential for conducting all experiments and for obtaining the final performance and usability results. By encapsulating every subsystem (API, MCP servers, LLM backends, agent API, and front-end) inside Docker services, the entire platform could be executed consistently on different hardware, operating systems, and LLM backends (local or remote). This standardization enabled controlled evaluations of latency, tool execution stability, and multi-agent orchestration across heterogeneous environments.

Terraform configuration played a critical role in validating that the system is deployable in real enterprise cloud environments. The modules were used to provision virtual machines, networking components, firewall rules, and other cloud resources across providers such as GCP, AWS, and Azure. This demonstrated that deploying a complete AI agent system, composed of microservices, MCP tool servers, and LLM runtimes, is technically feasible in distributed infrastructures with proper networking, isolation, and security considerations. The combination of Docker-based reproducibility and Terraform-based provisioning establishes a practical path for running the agent in both on-premise servers and cloud environments.

4.8 Tools, Libraries, and Technologies

The system was developed primarily using Python due to its strong ecosystem for machine learning, API development, and agent frameworks. The following tools and technologies formed the core implementation stack:

- **Programming languages:** Python (main logic), JSON (configuration), Bash scripts (automation), and Dockerfile (containerization).

- **Backend and API technologies:** FastAPI, Uvicorn, Motor (MongoDB async client), Python-Jose (JWT), Argon2-CFFI (password hashing), Pydantic (data validation), Email-Validator, and Requests.
- **Agent and LLM stack:** LangChain, LangChain-Community, LangChain MCP Adapters, Tiktoken, MCP (Model Context Protocol), Groq client, OpenAI client, and LangChain-Ollama.
- **Containerization and infrastructure:** Docker, Docker Compose, and Terraform for cloud provisioning.
- **Model hosting:** Support for both local and API-based execution of multiple LLMs.

A wide variety of LLM models were tested, including:

- **API-based models:** GPT-3.5, GPT-4, GPT-4o, GPT-4 Mini, GPT-5 Mini, LLaMA (Groq), Qwen-VL, DeepSeek-R1.
- **Local models (via Ollama):** LLaMA 3.2 3B, LLaMA 3.1 8B, LLaMA 3 1B, Mistral 7B, Dolphin 3 8B.

Evaluating such a heterogeneous model ecosystem was essential for comparing performance trade-offs involving:

- latency and throughput,
- operational cost,
- accuracy and hallucination rate,
- resource consumption,
- and privacy or data-safety requirements.

This evaluation directly supported one of the main goals of the system: enabling fully internal, private inference pipelines for enterprises dealing with sensitive data.

4.9 System Architecture and Design Strategy

The final design adopts a layered, modular architecture in which each component plays a well-defined functional role. The project repository follows the structure:

```
project-root/  
|-- agent/  
|   \-- multi/  
|-- api/  
|   |-- scripts/  
|   \-- src/  
|       |-- auth/  
|       |-- flights/  
|       \-- shared/  
|-- front/  
|-- infra/  
|-- mcp/  
|-- terraform/
```

The main architectural principles guiding the design were:

- **Strict isolation between the agent and the database:** Unlike early prototypes, the agent has **no direct access** to SQL or MongoDB. All operations flow through a validated API layer, eliminating injection risks.
- **Use of MCP (Model Context Protocol):** Tools are exposed through an MCP server using explicit schemas, capability negotiation, and structured communication. This prevents uncontrolled tool execution and makes agent behavior auditable.
- **API-first tool execution:** Tools (actions) are implemented inside the API as secure endpoints. The agent merely requests the action; it does not craft raw queries.
- **Full containerization:** The agent, MCP server, API, database, and front-end run in isolated containers, ensuring reproducibility and simplifying deployment.
- **Dual LLM capability:** The system is compatible with:
 - local LLMs (Ollama), enabling private inference;
 - cloud LLM APIs (OpenAI, Groq), enabling high-performance evaluation.
- **Cloud-ready infrastructure:** Terraform modules provision VMs, networking, firewall rules, and container registries across GCP, AWS, or Azure. This demonstrates the architectural feasibility of deploying enterprise-grade AI agents in distributed environments.

These principles directly address the limitations observed in early prototypes, particularly regarding security, fragility, and scalability, and guided the refinement of the final multi-agent architecture.

4.10 Evaluation Workflow and Lessons Learned

The final stage of project development consisted of evaluating how different LLMs behave when embedded in the multi-agent architecture. A benchmark dataset of 68 questions was created, covering tool-based workflows, RAG-style queries, and mixed planner decisions. Each model was tested using the same dataset to ensure experimental fairness.

Three primary metrics were measured:

- **Accuracy:** whether the agent selected and executed the correct MCP tool for each test case and produced an output consistent with the expected behavior of the API;
- **Latency:** total time required to process each question, including model inference and tool execution;
- **Resource usage:** CPU and memory consumption, especially when comparing local inference to API-based execution.

Correctness was evaluated manually based on expected tool execution behavior or the correctness of the information retrieved through RAG. Each LLM was executed once across the full benchmark due to time and resource constraints, but the single-run protocol was sufficient to reveal clear differences in behavior between models.

Experiments confirmed several practical trade-offs. API-based LLMs offered strong accuracy and robustness but introduced variable latency and recurring costs, as well as privacy concerns for sensitive datasets. Local models avoided these issues but required significant hardware resources; small models were fast but more error-prone, whereas larger models required GPU acceleration to reach acceptable latency.

Finally, the project reinforced the importance of security and engineering discipline in tool-enabled agents. The transition from direct database access to an API-first, MCP-mediated tool layer significantly reduced the attack surface and made the system easier to audit. However, building and maintaining MCP servers and tools still demands qualified engineering work: each tool must be carefully designed, validated, and monitored over time. These insights guided the design decisions reported in the subsequent chapters on results and discussion.

Chapter 5

Methodology

This chapter describes how the multi-agent MCP-based system was studied and evaluated. While the previous chapter detailed the engineering decisions and the development path, the focus here is on the methodological principles used to understand the architecture and compare different Large Language Models (LLMs) within it. The methodology combines conceptual analysis, iterative refinement, and controlled experimentation.

The chapter is organized into three components: the research and development approach, the evaluation strategy, and the metrics and data collection process.

5.1 Research and Development Approach

The methodological foundation follows a design science perspective, in which an artifact is created, justified, and evaluated in cycles. The artifact in this work is the final multi-agent architecture that integrates planners, FAQ agents, and MCP tool executors under consistent security and isolation guarantees.

Two research questions guided the process. RQ1 asks how MCP can support a modular and reliable multi-agent architecture. RQ2 asks how different LLMs behave inside this architecture in terms of accuracy, latency, and operational characteristics.

To address these questions, the project applied three methodological elements.

- **Conceptual grounding.** A review of existing approaches to tool use and agentic behavior, along with an analysis of MCP specifications, established the requirements for safe tool invocation, modular design, and explicit capability control.
- **Iterative refinement.** Several architectural versions were explored. Early prototypes helped identify concrete risks in direct tool execution and informed the transition to a fully isolated MCP-based design. The methodology does not revisit implementation details already covered in Chapter 4; it focuses instead on the logical progression toward a reliable architecture.

- **Production-inspired deployment.** The final system was deployed in a containerized environment that mimics realistic enterprise conditions. This ensures that the experiments reported in the next chapters reflect operationally meaningful behavior rather than toy examples or isolated components.

Within this framework, RQ1 is addressed through structural and qualitative evaluation of the architecture, and RQ2 is addressed through quantitative experiments designed to compare LLMs executing exactly the same workflow.

5.2 Evaluation Strategy

The evaluation examines how different LLMs operate when embedded in an identical multi-agent MCP architecture. All models interact with the same tools, prompts, and backend. Only the underlying LLM changes. This isolates the effect of the model on system behavior.

A benchmark dataset of 68 questions was created to represent typical workflows of an airline customer-service system. The scenarios include:

- FAQ queries handled by the FAQ sub-agent,
- basic tool calls such as searching for flights,
- multi-step flows requiring planning and sequential tool execution,
- negative or edge cases where refusal is the correct behavior.

Each model processes the same 68 prompts under the same system configuration. The goals are:

- to determine whether the architecture reliably guides the model toward correct tool use and consistent behavior (RQ1),
- to measure differences in accuracy, latency, and token usage across models (RQ2).

All experiments were executed in isolated containers to ensure that backend state, MCP behaviors, and system load were identical for every model.

5.3 Benchmark Question Categories

The benchmark of 68 questions was designed to cover the main interaction types that a multi-agent MCP-based system must handle. Each prompt was assigned to a category so that different abilities of the architecture and the LLMs could be evaluated in a controlled and comparable manner. The categories below summarize the structure of the dataset and its relevance to the study.

Query questions

These prompts request information such as listing flights, checking tickets, or retrieving baggage rules. They evaluate whether the agent selects the correct MCP tools when needed and relies on the FAQ agent when the answer is static. This category tests basic routing, tool selection, and correct API interaction.

Authentication questions

Prompts labeled as `login_auth` involve login attempts, session checks, or requests that implicitly require authentication. They test whether the agent recognizes when authentication is required, initiates it correctly, and respects session state. This is essential for evaluating safe workflow handling.

Action questions

These prompts request operations that modify state, such as buying or canceling a flight. They assess correct tool invocation, parameter handling, and the ability to follow business rules. Negative but justified outcomes, such as refusing an invalid purchase, are treated as correct.

Ambiguous questions

These prompts do not contain enough information for direct action. They evaluate intent recognition and the agent’s ability to avoid unnecessary or incorrect tool calls. This category highlights how well each model handles vague or incomplete inputs.

Stress-test questions

These scenarios combine several steps, conditional logic, or conflicting objectives. They test planning, tool sequencing, and multi-step reasoning under higher complexity. They are particularly useful for comparing the limits of local and API-based models.

5.4 Metrics and Data Collection

Each model answered all 68 benchmark questions. For every question, the system logged three primary metrics: functional correctness, latency, and token usage.

5.4.1 Batch Execution and Logging Pipeline

A batch script executed the benchmark. For each question, the script reset the internal metrics of the multi-agent orchestrator and invoked `handle_message(prompt)`.

Latency was measured with `time.perf_counter()` and reflects the complete end-to-end duration of planning, tool invocation through MCP, and response generation.

Token usage was recorded using the orchestrator’s internal meter. When the meter was not available, a fallback approximation based on whitespace tokenization of the prompt and reply was used.

Each record stored:

- the model and provider,
- the user prompt and truncated reply,
- end-to-end latency in milliseconds,
- input, output, and total tokens,
- estimated monetary cost when applicable,
- the last MCP tool invoked by the agent.

All results were saved in JSONL format. This procedure was repeated for each model, ensuring strict comparability.

5.4.2 Accuracy Definition and Annotation Procedure

Accuracy required a definition aligned with realistic system behavior. A correct answer is not necessarily a positive outcome. In some scenarios, the model must refuse an invalid request. For example, denying the purchase of a flight that the user already owns is considered correct.

Accuracy was computed using a hybrid annotation pipeline that combined heuristic filters and a verification LLM.

1. **Heuristic detection of clear errors.** Replies containing generic failure messages, execution errors, or inability to match requests to tools were flagged as incorrect unless they matched a valid negative outcome defined by the business rules.
2. **LLM-based semantic verification.** A dedicated verification model received the user prompt and the agent reply and produced a binary decision: true if the answer satisfied the request, false otherwise. The verifier was instructed to treat justified negative outcomes as correct.
3. **Final accuracy assignment.** Each interaction was labeled with 1 for correct and 0 for incorrect. If the verifier could not decide, the interaction was conservatively marked as incorrect. The accuracy of a model corresponds to the mean of these labels over the 68 executions.

This method captures both correct tool-mediated actions and correct refusals, while also detecting cases where a model selects an inappropriate tool or produces hallucinated behavior.

5.4.3 Aggregation of Latency and Token Metrics

After all executions, the analyzer grouped the logs per model and computed aggregated metrics. Latencies were separated into interactions that involved MCP tool calls and those that did not. For each model, the following values were calculated:

- average latency with MCP and without MCP,
- average input, output, and total tokens,
- final accuracy using the definition in Section 5.4.2.

These aggregated metrics support the comparisons presented in the Results and Discussion chapters. They allow the analysis of performance trade-offs and functional reliability in a realistic enterprise environment.

Chapter 6

Results

This chapter presents the results obtained from evaluating the multi-agent MCP-based system using both local and API-based Large Language Models (LLMs). All models were evaluated using the same benchmark of 68 questions, covering tool-based workflows, FAQ-style interactions, and mixed planner decisions. Metrics collected include latency (with and without MCP tool usage), accuracy, and average token consumption. To improve readability, tables are divided into smaller groups distinguishing local and API-based models.

6.1 Local LLM Results

Local models were executed using Ollama in a CPU-only environment. This configuration represents organizations that prioritize data privacy, on-premise inference, and full control over the execution stack.

Latency and Accuracy

Table 6.1: Latency and accuracy for local LLMs.

Model	MCP Latency (ms)	FAQ Latency (ms)	Accuracy
llama3.2 3B	28721.02	20714.95	0.91
llama3 8B	75141.70	84363.80	0.82
qwen2.5 7B	69486.44	59804.35	0.88

Token Usage

Table 6.2: Token statistics for local LLMs.

Model	Input Tokens	Output Tokens	Total Tokens
llama3.2 3B	1994.46	116.31	2110.76
llama3 8B	2432.78	120.01	2552.79
qwen2.5 7B	2234.84	104.49	2339.32

Summary of Local Performance. Local inference produced the highest latencies, reaching 20–80 seconds per interaction. Despite this, accuracy remained competitive, frequently above 0.85. The llama3.2 3B model delivered the best balance between speed and correctness. Latency differences between MCP and FAQ modes were small relative to the inherent slowness of CPU-bound inference.

6.2 API-Based LLM Results

API-based models represent high-performance remote inference relying on cloud infrastructure optimized for large-scale deployments.

Latency and Accuracy

Table 6.3: Latency and accuracy for API-based LLMs.

Model	MCP Latency (ms)	FAQ Latency (ms)	Accuracy
llama-3.1 8B Instant	913.35	841.60	0.94
qwen3 32B	1724.91	2775.50	0.75
llama-3.3 70B	1014.94	890.80	0.91
gpt-4o-mini	2356.75	4115.50	0.97
gpt-4-turbo	3713.87	6072.70	0.99
gpt-5-mini	13176.05	14233.50	0.88

Token Usage

Table 6.4: Token statistics for API-based LLMs.

Model	Input Tokens	Output Tokens	Total Tokens
llama-3.1 8B Instant	2093.72	88.63	2182.35
qwen3 32B	1593.62	487.37	2080.99
llama-3.3 70B	2625.41	80.66	2706.07
gpt-4o-mini	2118.72	66.65	2185.37
gpt-4-turbo	2097.07	73.97	2171.04
gpt-5-mini	2269.59	648.53	2918.12

Summary of API-Based Performance. API-based models consistently outperformed local ones in latency, typically responding within 1–4 seconds, except for the larger GPT-5 Mini model. Accuracy reached up to 0.99 for GPT-4 Turbo and 0.97 for GPT-4o Mini. Models such as qwen3-32B showed weaker accuracy and higher output token usage, which increased latency.

6.3 Cost and Resource Considerations

Costs for API-based models follow their provider pricing (Table 2.1), whereas local models incur hardware and energy-related expenses. The results suggest that API-based LLMs offer significant performance advantages per dollar for low-to-medium workload volumes, while local models become cost-effective only at large scale or under strict privacy requirements.

6.4 General Observations

Overall, the experiments reveal that MCP introduces negligible latency overhead compared to FAQ-only responses, and in certain models (e.g., GPT-4o Mini, GPT-4 Turbo), MCP usage even decreased latency relative to FAQ mode, primarily due to lower input token requirements when invoking structured tools.

Chapter 7

Discussion

This chapter synthesizes the findings presented in Chapter 6 and evaluates how model choice, deployment environment, and orchestration mechanisms affect the behavior of the MCP-based system. The discussion focuses on the reliability added by MCP, the relationship between latency and token usage, the feasibility of fully local deployments, cost and privacy implications, and the broader consequences for designing systems that rely on structured tool execution.

7.1 MCP and Reliability in Tool Execution

The adoption of the Model Context Protocol created a stable boundary between language models and operational components. All interactions with the FlightCo backend, including authentication and flight management operations, were executed through MCP tools with explicitly defined schemas. This enforced structure ensured predictable behavior and allowed each tool to be audited and validated before execution.

A central result of this project is that it is technically feasible to build an entire tool ecosystem from scratch and operate it fully locally. The system included a custom REST API, a local MCP server, a MongoDB database, and local LLM inference. All components ran on a machine provided by the IME-USP Linux network. Although this hardware was not optimized for LLM inference, it played an essential role by making it possible to test the boundaries of a complete local deployment. The experiments confirm that MCP functions consistently in a closed environment and that a fully local multi-agent system is practical as long as computational constraints are understood.

The same architecture also supports deployments that rely entirely on remote APIs. Since MCP abstracts how each tool is implemented, the agent layer interacts with local or remote servers through identical interfaces. This shows that MCP does not only standardize tool invocation but also supports flexible deployment decisions.

7.2 Latency Differences and Token Effects

The results reveal substantial latency differences between local and API-based inference. Local models running through Ollama in a CPU-only environment consistently required 20 to 80 seconds per interaction. This behavior is expected in settings where large context windows and multi-step reasoning must be computed without GPU acceleration. Even under these constraints, accuracy remained competitive, which suggests that correctness is not inherently tied to hardware performance.

By contrast, API-based models provided responses in one to four seconds, benefiting from specialized cloud infrastructure. A notable observation is that for several API models, MCP mode was not slower than FAQ mode. In some cases it was faster because FAQ prompts required embedding curated text and additional metadata, increasing token counts and therefore inference time. MCP prompts, on the other hand, relied on compact structured tool definitions.

Latency for local models could be significantly improved by upgrading hardware. Modern GPUs optimized for inference, faster CPUs, and larger memory bandwidth would reduce processing time for the same multi-agent prompts. The project demonstrates that the architecture remains stable under limited hardware and that latency improvements are primarily a matter of computational resources rather than system design. This indicates a clear path for future work: exploring specialized hardware to enable competitive performance in fully local deployments.

Token usage also influenced latency. Multi-agent orchestration increases context size and therefore affects both cost (for API models) and response time (for all models). Any optimization in prompt construction, context trimming, or compression strategies would have immediate benefits in both local and remote environments.

7.3 Cost, Privacy, and Viability of Local Deployment

The performance differences described above have direct cost and privacy implications. API-based LLMs deliver high accuracy and low latency without requiring organizations to maintain expensive hardware. This makes them attractive for general-purpose deployments without strict confidentiality requirements.

Local deployments invert the trade-off. Running the entire pipeline within a private environment eliminates external data transmission and gives organizations full control over the system. The experiments show that even modest local hardware can support a fully functioning MCP-based system, although with high latency. For enterprises where privacy is a regulatory or operational requirement, this trade-off may be acceptable. Furthermore, once hardware is upgraded, marginal inference cost approaches zero, making local deployments economically viable at large scale.

The architecture implemented in this thesis supports both strategies. Tools can be mapped to local or remote MCP servers without modifying the agent logic, enabling hybrid deployments where sensitive operations remain on-premise and non-sensitive or high-throughput tasks are offloaded to cloud providers.

7.4 Implications for Multi-Agent Design

The multi-agent architecture used in this work was designed to separate planning, tool execution, and FAQ handling. This separation improved clarity during development and made the system easier to debug and extend. At the same time, multi-agent orchestration increases token usage because each agent introduces additional context. This cost is inherent to structured workflows that rely on explicit reasoning steps.

Planner decisions had a strong impact on overall system quality. Incorrect routing, such as selecting FAQ instead of a tool, led to incomplete responses even when tools were functioning correctly. Improving planner reliability is therefore an important direction for future work. Potential enhancements include context reduction, more compact prompting strategies, specialized planners using smaller models, caching of tool outputs, and adaptive routing based on recent performance.

Overall, the discussion reinforces that MCP enables a modular and flexible architecture in which all components can run locally, remotely, or in any hybrid configuration. The experiments conducted on the IME-USP Linux infrastructure demonstrate that a fully local system is not only possible but operational, and that meaningful performance improvements can be achieved by investing in hardware designed for LLM workloads.

Chapter 8

Conclusion

This work investigated the development of a multi-agent conversational system built on the Model Context Protocol (MCP), integrating both local and API-based Large Language Models (LLMs). The project demonstrated that it is technically feasible to design and operate agents capable of planning, selecting tools, and executing actions through a unified protocol. The final architecture combined a custom API, an MCP tool server, and a coordinated set of planner, executor, and FAQ agents, showing that heterogeneous components can be connected reliably within a controlled framework.

The experiments confirm that MCP supports modularity, extensibility, and clear execution boundaries, enabling consistent interaction between agents and external systems. At the same time, the results highlight practical limitations. Token usage remained high for all evaluated models, which increases operational cost when using API-based inference. Latency was also significant, especially in local environments, where average responses occurred on the order of tens of seconds. This restricts the use of such systems in time-sensitive applications. In addition, the models did not always select the correct tool or reasoning path, which indicates areas where current LLM behavior and MCP workflows still require refinement.

Despite these constraints, the evaluation shows that MCP-based multi-agent systems are viable for scenarios where strict real-time performance is not required. They provide a structured and auditable way to combine different LLMs, connect multiple MCP servers, and coordinate agents across diverse workflows. This makes the approach suitable for a variety of industrial applications and for internal automation tasks that benefit from controlled and well-defined interactions with external systems.

In conclusion, the project shows that developing multi-agent architectures with MCP and multiple LLMs, whether local or remote, is feasible and practical with current technologies. Although improvements in latency, cost efficiency, and tool-selection reliability are still necessary, the architecture presented here establishes a solid foundation for future systems that require safe, modular, and interoperable AI-driven operations.

Bibliography

- [1] “Llm price check.” <https://llmpricecheck.com/>. Accessed: 2025-11-19.
- [2] P. P. Ray, “A survey on model context protocol: Architecture, state-of-the-art, challenges and future directions,” *arXiv preprint arXiv:2503.00000*, 2025. Accessed: 2025-12-03.
- [3] M. Raza, Z. Jahangir, M. B. Riaz, M. J. Saeed, and M. A. Sattar, “Industrial applications of large language models,” *Scientific Reports*, vol. 15, no. 1, p. 13755, 2025.
- [4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2023. ICLR 2023 (v3).
- [5] J. Irungu, S. Graham, A. Girma, and T. Kacem, “Artificial intelligence techniques for sql injection attack detection,” in *Proceedings of the 2023 8th International Conference on Intelligent Information Technology (ICIIT)*, (Da Nang, Vietnam), ACM, 2023.
- [6] M. de Oliveira Costa and M. A. Loureiro, “Prompt security: A survey on threats and defense mechanisms in llm-based systems,” *Preprints*, 2024. Preprint.
- [7] Y. Chen, Q. Ye, X. L. Li, V. Zhong, Y. Xu, C. D. Manning, D. Chen, T. Hashimoto, and N. A. Smith, “Agentic behavior in large language models: An empirical survey,” *arXiv preprint arXiv:2407.10972*, 2024.
- [8] HatchWorks, “Open source vs closed llms: Everything you need to know.” <https://hatchworks.com/blog/gen-ai/open-source-vs-closed-llms-guide/>, 2024. Accessed: 2025-11-16.
- [9] S. Intelligence, “Conversational ai explained: Top 9 tools & how-to guide (including gpt).” <https://spotintelligence.com/2024/01/30/conversational-ai-explained-top-9-tools-how-to-guide-including-gpt/>, 2024. Accessed: 2025-11-16.

-
- [10] Hostinger, “10 best ai automation tools to boost productivity in 2024.” <https://www.hostinger.com/tutorials/best-ai-automation-tools>, 2024. Accessed: 2025-11-16.
 - [11] AIML.com, “Explain the transformer architecture.” <https://aiml.com/explain-the-transformer-architecture/>, 2024. Accessed: 2025-11-16.
 - [12] Descope, “Model context protocol (mcp) — the new language for agents.” <https://www.descope.com/learn/post/mcp>, 2024. Accessed: 2025-11-16.
 - [13] X. Hou, Y. Zhao, S. Wang, and H. Wang, “Model context protocol (mcp): Landscape, security threats, and future research directions,” *arXiv preprint arXiv:2503.23278*, 2025.